

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC - KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỔNG HỢP_HƯỚNG TRÍ TUỆ NHÂN TẠO

Đề tài:

Nhận diện vật nuôi sử dụng mạng nơron tích chập

GVHD: Vũ Văn Tiến

STT	Họ tên SV	MSSV
1	Nguyễn Anh Kiệt	2211758
2	Nguyễn Tuấn Kiệt	2211765
3	Dương Gia Lâm	2211806
4	Nguyễn Anh Lâm	2211795
5	Hoàng Nhật Linh	2211847

TP. Hồ Chí Minh, Tháng 9/2024

Mục lục

1	LÝ DO CHỌN ĐỀ TÀI	3
1.1	Vấn đề xã hội	3
1.2	Vấn đề kĩ thuật	3
2	MỤC TIÊU ĐỀ TÀI	4
3	PHƯƠNG PHÁP NGHIÊN CỨU	5
3.1	Neural Network	5
3.1.1	Neural Network là gì?	5
3.1.2	Mô hình Neural Network nhân tạo (ANN) và Feedforward	5
3.1.3	Cost và Loss Function	8
3.1.4	Backpropagation, Gradient Descent và Learning rate	9
3.1.5	Một số Activation Function	15
3.2	Convolutional Neural Network(CNN)	18
3.2.1	Xử lý ảnh trong máy tính	18
3.2.2	Phép tính chập (Convolution)	18
3.2.3	Convolutional Neural Network	19
3.2.4	Các metrics trong bài toán phân loại	24
3.3	Một số mô hình đề xuất	24
3.3.1	Mạng VGG 16	24
3.3.2	Mạng ResNet	26
4	Chọn model để hiện thực	31
4.1	Phân tích dataset	31
4.2	Xử lý dataset bằng phương pháp Undersampling	32
4.3	Tiền xử lý và Tăng cường Dữ liệu (Data Preprocessing and Augmentation)	33
5	HIỆN THỰC MÔ HÌNH	35
5.1	Hướng tiếp cận với pretraining (Sử dụng mô hình pretrain)	35
5.2	Hướng tiếp cận không sử dụng pretraining (Không sử dụng mô hình pretrain)	38
6	ĐÁNH GIÁ	41
7	PHƯƠNG HƯỚNG	43
8	TÀI LIỆU THAM KHẢO	43



Danh sách phân chia công việc

No.	Fullname	Student ID	Problems	Percentage of work
1	Nguyễn Anh Kiệt	2211758	- Xây dựng model CNN - Code và hiện thực model	20%
2	Nguyễn Tuấn Kiệt	2211765	- Tìm hiểu lý thuyết ANN - Tìm hiểu model VGG16 và code hiện thực - Code Latex	20%
3	Dương Gia Lâm	2211806	- Tìm hiểu lý thuyết ANN - Đóng góp xây dựng model	20%
4	Nguyễn Anh Lâm	2211795	- Soạn báo cáo - Viết code Latex - Tìm hiểu model VGG16 và hiện thực	20%
5	Hoàng Nhật Linh	2211847	- Hỗ trợ xây dựng model CNN - Làm slide thuyết trình - Tìm kiếm và thu thập tài liệu CNN	20%

1 LÝ DO CHỌN ĐỀ TÀI

1.1 Vấn đề xã hội

Các vườn quốc gia và khu bảo tồn thiên nhiên có diện tích rất rộng lớn và thường nằm ở những khu vực có địa hình phức tạp hoặc vùng sâu, vùng xa. Điều này tạo ra nhiều thử thách trong việc giám sát và quản lý động vật, đặc biệt là đối với các loài hoang dã quý hiếm, vốn không dễ dàng quan sát và theo dõi. Việc tiếp cận các khu vực này đòi hỏi thời gian, chi phí cao, và nguồn nhân lực lớn, gây khó khăn trong việc triển khai các biện pháp giám sát và bảo vệ các loài động vật 1 cách hiệu quả.

Nhiều loài động vật quý hiếm đang bị đe dọa nghiêm trọng bởi các yếu tố như săn bắt trái phép, mất môi trường sống do sự phát triển của đô thị hóa, nông nghiệp, hoặc do biến đổi khí hậu, thiên tai. Những loài này thường cần môi trường sống đặc biệt và không thể tồn tại lâu dài nếu không có sự bảo vệ đúng mức. Việc bảo vệ chúng đòi hỏi một hệ thống giám sát liên tục và có khả năng phát hiện các mối đe dọa kịp thời để can thiệp nhanh chóng.

Việc thu thập và phân tích dữ liệu về động vật hoang dã truyền thống thường gặp nhiều khó khăn. Các phương pháp quan sát thủ công tốn nhiều thời gian, nhân công, và dễ bị sai sót hơn dẫn đến thiếu thông tin chính xác về các quần thể động vật và tình trạng của chúng.

1.2 Vấn đề kĩ thuật

Nhìn nhận thấy vấn đề này, trước đây, trong lĩnh vực thị giác máy tính, các phương pháp học máy cổ điển đã được phát triển và áp dụng để giải quyết các bài toán nhận diện hình ảnh, bao gồm cả việc phân loại động vật trong các khu bảo tồn. Tuy nhiên, các công nghệ này dần bộc lộ những hạn chế lớn về độ chính xác và khả năng xử lý các tình huống phức tạp. Các mô hình học máy truyền thống thường không đủ mạnh để xử lý các vấn đề như nhận diện động vật trong các điều kiện ánh sáng yếu, môi trường thay đổi liên tục, hoặc sự tương đồng giữa các loài động vật. Chính vì vậy, các công nghệ này ngày càng không đáp ứng được yêu cầu của thực tế trong công tác giám sát và bảo vệ động vật hoang dã.

Trong khi đó, Convolutional Neural Network (CNN) - một thuật toán đặc trưng của học sâu (Deep Learning) - đã chứng tỏ được ưu thế vượt trội trong các bài toán thị giác máy tính. CNN có khả năng học và biểu diễn các đặc trưng hình ảnh ở nhiều mức độ khác nhau nhờ vào cấu trúc mạng nơ-ron sâu, điều này cho phép nó nhận diện và phân loại các đối tượng trong hình ảnh một cách chính xác và hiệu quả hơn rất nhiều so với các phương pháp học máy cổ điển.

CNN đặc biệt hiệu quả trong việc xử lý hình ảnh phức tạp và không đồng nhất, điều này rất phù hợp với các bài toán bảo tồn động vật, nơi các loài động vật có thể xuất hiện trong các điều kiện hình ảnh khác nhau, như ánh sáng yếu, bị che khuất, hoặc ở các góc nhìn không thuận lợi. Các lớp tích chập trong mạng CNN có khả năng tự động trích xuất đặc trưng hình ảnh mà không cần phải xây dựng các bộ lọc thủ công, từ đó giảm thiểu các sai sót trong quá trình phân loại. Dựa trên các ưu điểm và tiềm năng ứng dụng của công nghệ CNN, một loạt các mô hình CNN nổi trội đã được nghiên cứu và phát triển, nổi bật nhất có thể kể đến như AlexNet, ResNet và VGG.

AlexNet : AlexNet là một trong những mô hình CNN nổi tiếng đầu tiên và có ảnh hưởng lớn đến sự phát triển của deep learning trong nhận diện hình ảnh. Được thiết kế bởi Alex Krizhevsky, Ilya Sutskever và Geoffrey Hinton vào năm 2012. AlexNet đã giành chiến thắng trong cuộc thi ImageNet với một cách biệt rất lớn so với các phương pháp trước đó. Mô hình này có cấu trúc gồm 8 lớp, bao gồm các lớp tích chập và lớp kết nối đầy đủ (fully connected layers), và sử dụng hàm kích hoạt Rectified Linear Unit (ReLU) giúp tăng tốc quá trình huấn luyện. AlexNet đã chứng minh rằng việc sử dụng mạng sâu có thể cải thiện đáng kể độ chính xác trong các bài toán phân loại hình ảnh, mở ra kỷ nguyên của deep learning trong thị giác máy tính. Tuy nhiên, mặc dù rất hiệu quả trong nhận diện hình ảnh, AlexNet có một số hạn chế như việc sử dụng mô hình quá sâu và yêu cầu phần cứng mạnh mẽ để huấn luyện, điều này đôi khi làm giảm tính khả thi của việc triển khai nó trong các môi trường tính toán hạn chế.

ResNet (Residual Networks) : ResNet, do Microsoft Research phát triển, là một trong những mô hình CNN đột phá nhất trong những năm gần đây. ResNet nổi bật với kiến trúc mạng

neuron dư thừa (residual network), trong đó sử dụng các Residual Blocks để kết nối các lớp không liên tiếp với nhau thông qua các kết nối nhảy (skip connections). Kiến trúc này giúp giải quyết vấn đề của các mạng sâu, đó là hiện tượng "vanishing gradient" (biến mất gradient) khi huấn luyện mạng với nhiều lớp. Bằng cách cho phép tín hiệu được truyền qua các lớp sâu mà không bị suy giảm, ResNet có thể dễ dàng huấn luyện các mô hình cực kỳ sâu (với hàng trăm lớp) mà không gặp phải vấn đề phổ biến ở các mạng neuron truyền thống. Mô hình ResNet đã đạt được thành tích xuất sắc trong các cuộc thi nhận diện hình ảnh, đặc biệt là cuộc thi ImageNet, và hiện nay đang được sử dụng rộng rãi trong nhiều ứng dụng thực tế, từ nhận diện đối tượng đến phân tích hình ảnh y tế.

VGG (Visual Geometry Group) : VGG là một mô hình CNN được phát triển bởi nhóm nghiên cứu tại Đại học Oxford, nổi bật với cấu trúc đơn giản và dễ hiểu. Mô hình này được biết đến với tên gọi VGG16 và VGG19, tùy thuộc vào số lớp trong mạng. VGG đặc trưng bởi việc sử dụng các lớp tích chập với bộ lọc có kích thước nhỏ (3×3) thay vì các bộ lọc lớn như trong các mô hình trước đó. Điều này giúp giảm số lượng tham số cần học, đồng thời tăng cường khả năng học được các đặc trưng chi tiết của hình ảnh. VGG mang lại hiệu suất cao trong nhiều bài toán nhận diện hình ảnh và phân loại đối tượng, nhưng nhược điểm lớn của mô hình này là số lượng tham số rất lớn, điều này dẫn đến yêu cầu phần cứng rất cao trong quá trình huấn luyện và triển khai.

Các mô hình CNN như AlexNet, VGG, và ResNet đều có những ưu điểm đáng kể nhưng cũng tồn tại một số hạn chế. Các mô hình này đều cho phép phân loại hình ảnh chính xác và hiệu quả, nhưng mỗi mô hình có đặc điểm riêng:

- AlexNet : dễ dàng huấn luyện và có hiệu suất tốt trên các bài toán phân loại hình ảnh cơ bản, nhưng không đủ mạnh mẽ cho các bài toán phức tạp hơn, lại có yêu cầu cao về phần cứng để triển khai thực hiện.
- VGG : cấu trúc đơn giản và dễ hiểu, nhưng có số lượng tham số rất lớn, mất nhiều thời gian tính toán và chiếm nhiều bộ nhớ để lưu trữ.
- ResNet : hiệu quả trong việc huấn luyện các mạng cực kỳ sâu nhờ vào các residual blocks, nhưng có thể phức tạp trong việc triển khai và yêu cầu phần cứng mạnh.

Với những ưu điểm nổi bật này, CNN hiện đang được áp dụng rộng rãi trong lĩnh vực thị giác máy tính, không chỉ trong nhận diện động vật mà còn trong nhiều ứng dụng khác như nhận diện khuôn mặt, phân tích hình ảnh y tế, và nhận diện các vật thể trong môi trường tự nhiên. Tuy nhiên, dù CNN có tiềm năng lớn, việc triển khai công nghệ này vào công tác bảo tồn động vật hoang dã vẫn còn gặp phải một số thách thức kỹ thuật, đặc biệt là trong việc thu thập và xử lý dữ liệu hình ảnh thực tế từ các khu bảo tồn, nơi điều kiện môi trường có thể ảnh hưởng đáng kể đến chất lượng hình ảnh thu thập được.

2 MỤC TIÊU ĐỀ TÀI

Mục tiêu chung của đề tài là ứng dụng công nghệ mạng neuron tích chập (CNN) để phân loại động vật, hỗ trợ công tác quản lý và bảo tồn các loài trong các khu bảo tồn thiên nhiên. Việc phân loại chính xác hình ảnh động vật thông qua CNN không chỉ giúp theo dõi số lượng và sự hiện diện của từng loài mà còn tạo điều kiện thuận lợi cho việc mở rộng áp dụng công nghệ này ở nhiều khu bảo tồn khác nhau. Nghiên cứu nhằm xây dựng một nền tảng tự động và hiệu quả, giúp các nhà bảo tồn dễ dàng phân tích và đưa ra quyết định quản lý dựa trên dữ liệu về tần suất và phân bố của động vật, từ đó bảo vệ và duy trì đa dạng sinh học.

Bên cạnh việc phân loại hình ảnh động vật, công nghệ CNN còn có thể hỗ trợ dự đoán phân bố của các loài trong các khu vực khác nhau dựa trên tần suất xuất hiện của chúng. Bằng cách thu thập dữ liệu về số lần và vị trí các loài động vật được phát hiện, hệ thống có thể cung cấp thông tin về sự phân bố tự nhiên của từng loài trong khu bảo tồn. Điều này giúp các nhà quản lý có thể theo dõi sự thay đổi của động vật theo thời gian và không gian, từ đó nắm bắt xu hướng

di cư, phân bố, hoặc các dấu hiệu bất thường trong hành vi của chúng, hỗ trợ công tác bảo tồn và bảo vệ môi trường sống của từng loài.

Việc theo dõi sự thay đổi phân bố loài theo thời gian là một bước quan trọng trong việc nhận diện các dấu hiệu di cư bất thường của động vật. Thông qua phân tích sự dịch chuyển của các loài theo mùa hoặc theo chu kỳ, hệ thống CNN có thể phát hiện những biến đổi đáng chú ý trong phân bố và đưa ra cảnh báo về các hiện tượng không thường xuyên. Những bất thường này có thể là dấu hiệu của các vấn đề nghiêm trọng như sự can thiệp của con người (nạn săn bắt trộm), thiên tai (bão lũ, hạn hán), ô nhiễm môi trường hoặc hiện tượng xói mòn, sạt lở làm ảnh hưởng đến nơi sinh sống của động vật. Với những thông tin này, các nhà quản lý có thể điều tra nguyên nhân cụ thể, đề xuất các biện pháp khắc phục, và triển khai các giải pháp bảo vệ môi trường sống tự nhiên của động vật, góp phần nâng cao hiệu quả bảo tồn.

Trong phạm vi đề tài nghiên cứu này, mục tiêu chính là tìm hiểu về công nghệ CNN, tìm hiểu về các công nghệ và mô hình nền tảng, dựa vào đó để hiện thực nên mô hình CNN có khả năng phân biệt hình ảnh động vật, xác định loài và đánh dấu các trường hợp không thuộc các loài đã định sẵn để kiểm tra thủ công. Dù có thể mở rộng công nghệ này để ứng dụng vào các mục tiêu quản lý động vật rộng hơn, đề tài này tập trung vào việc nghiên cứu tính năng phân loại hình ảnh các loài động vật, tạo nền tảng kỹ thuật cho các ứng dụng trong tương lai mà không đi sâu vào dự đoán phân bố hay theo dõi di cư của các loài.

3 PHƯƠNG PHÁP NGHIÊN CỨU

3.1 Neural Network

3.1.1 Neural Network là gì?

Một trong những điều kỳ diệu của thế giới tự nhiên là khả năng phân biệt đối tượng của động vật. Chó, chẳng hạn, có thể dễ dàng phân biệt chủ nhân với người lạ, trong khi trẻ sơ sinh đã có thể nhận biết khuôn mặt của mẹ ngay từ những ngày đầu đời. Những khả năng tưởng chừng đơn giản này lại là một bài toán phức tạp đối với trí tuệ nhân tạo (AI). Vậy đâu là nguyên nhân của sự khác biệt này?

Câu trả lời nằm ở cấu trúc phức tạp của não bộ, nơi hàng tỷ nơ-ron thần kinh kết nối với nhau tạo thành một mạng lưới thông tin khổng lồ. Mạng lưới này cho phép não bộ thực hiện vô số nhiệm vụ, từ việc nhận biết hình ảnh, âm thanh đến việc học hỏi và suy nghĩ.

Từ việc nhận thấy khả năng phân biệt của động vật, các nhà khoa học đã phát triển một mô hình tính toán gọi là mạng nơ-ron nhân tạo (Neural Network - NN). NN là một hệ thống tính toán được lấy cảm hứng từ cấu trúc và hoạt động của các nơ-ron sinh học. Bằng cách mô phỏng cách các nơ-ron kết nối và truyền thông tin, NN có thể học hỏi từ dữ liệu và thực hiện các nhiệm vụ phức tạp như nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên và thậm chí là ra quyết định.

Tuy nhiên, việc xây dựng một NN mô phỏng đầy đủ chức năng của não bộ vẫn còn là một thách thức lớn. Các nhà khoa học vẫn đang không ngừng nghiên cứu để cải tiến kiến trúc và thuật toán của NN, nhằm tăng cường khả năng học hỏi, thích ứng và giải quyết các vấn đề phức tạp hơn.

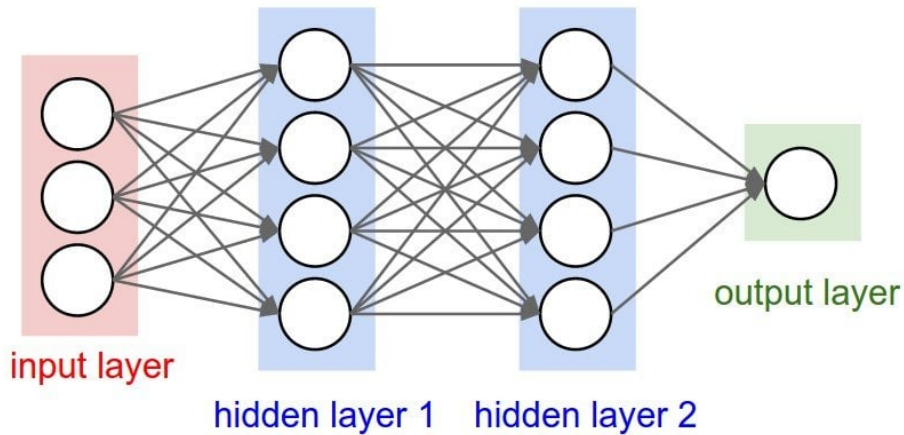
3.1.2 Mô hình Neural Network nhân tạo (ANN) và Feedforward

3.1.2.1 Mô hình ANN tổng quát

Hình trên là một mạng Neuron đơn giản.

Layer đầu tiên là Input, các layer ở giữa gọi là Hidden layer và lớp layer cuối cùng được gọi là Output player.

Ở mỗi Layer, các hình tròn được gọi là node hoặc unit. Mỗi node có một hệ số Bias (b), các node ở lớp phía trước liên kết với các node phía sau đó bằng các đường thẳng có trọng số weight (w) và được gọi là Edge.



Hình 1: Mô hình ANN tổng quát

Mỗi mô hình NN đều luôn có 1 Input layer và 1 output layer, có thể có hoặc không có hidden layer. Tổng số lớp trong mô hình được qui ước bằng tổng số layer – 1 (tức là không tính input layer).

Ví dụ: mô hình trên có 1 input layer, 2 hidden layer và 1 output layer => Tổng số layer trong mô hình là 3.

Ngoài ra, các node được tính dựa trên các b và w trước đó và diễn ra theo hai bước tính tổng Linear (được biểu diễn rõ ở mục sau) và áp dụng hàm kích hoạt (tìm hiểu ở mục 3.1.5).

Vì sao cần có hệ số bias (b) cho mỗi node ?

Hệ số Bias đóng vai trò vô cùng quan trọng trong việc quyết định điểm hoạt động của một neuron trong mạng neural network. Nó cho phép mạng neural network có khả năng biểu diễn một không gian đặc trưng rộng lớn hơn, từ đó nâng cao khả năng học tập và dự đoán.

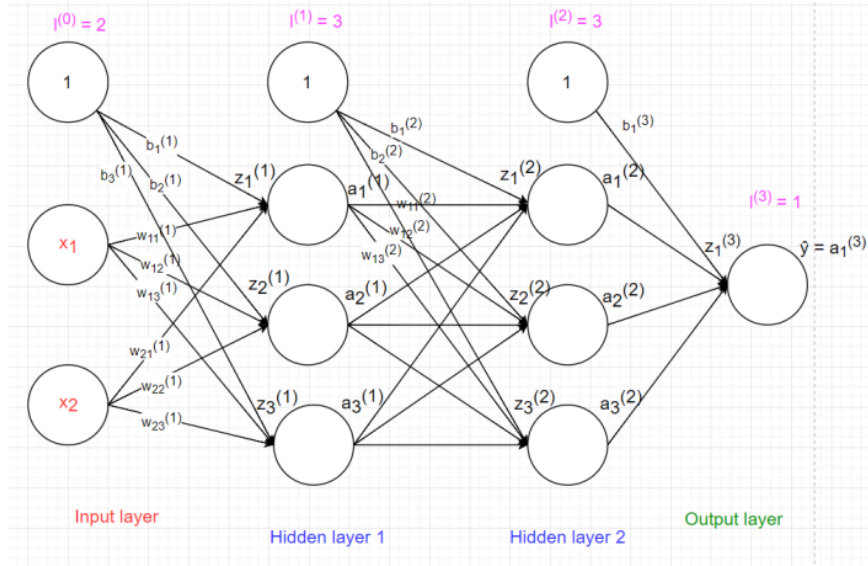
Thực ra ta xét 1 ví dụ đơn giản, trong mô hình Hồi qui tuyến tính (Linear Expression), bias đóng vai trò là hệ số tự do trong phương trình $y = w \cdot x + b$; nếu $b = 0$ tức là $y = w \cdot x$ thì đường thẳng này luôn qua gốc tọa độ. Điều này có nghĩa là nơ-ron chỉ có thể kích hoạt khi tổng giá trị đầu vào nhân với trọng số lớn hơn một ngưỡng nhất định (threshold). Nếu có hệ số b , thì phương trình sẽ dịch chuyển lên xuống 1 lượng b . Điều này giúp nơ-ron có thể kích hoạt ở nhiều mức độ khác nhau, không chỉ khi tổng giá trị đầu vào đạt đến một ngưỡng cố định. Nên giá trị bias rất quan trọng trong việc biểu diễn các hàm tuyến tính hoặc phi tuyến phức tạp hơn.

3.1.2.2 Feedforward

Xét mô hình neuron trên gồm 3 layer, Input layer có 2 node, hidden layer 1 có 3 node và hidden layer 2 có 3 node, output layer có 1 node.

Các kí hiệu:

- x_i : dữ liệu input.
- $b_i^{(l)}$: bias thứ i của lớp l ($i, l \geq 1$).
- $w_{ij}^{(l)}$: trọng số w từ node i của lớp $l - 1$ đến node j của lớp l ($i, l \geq 1$)
- $z_i^{(l)}$: Tổng linear của node i ở lớp l tính bằng cách nhân giá trị node ở layer $l - 1$ với hệ số w tương thích rồi cộng với b .
- $a_i^{(l)}$: Áp dụng hàm activate của node thứ i ở lớp thứ l ($i, l \geq 1$).



Hình 2: Ví dụ 1

Do mỗi node trong hidden layer và output layer đều có bias nên trong input layer và hidden layer cần thêm node 1 để tính bias (nhưng không tính vào tổng số node layer có).

Tại Hidden layer 1:

- $z_1^{(1)} = x_1^{(0)} * w_{11}^{(1)} + x_2^{(0)} * w_{21}^{(1)} + b_1^{(1)}$
- $z_2^{(1)} = x_1^{(0)} * w_{12}^{(1)} + x_2^{(0)} * w_{22}^{(1)} + b_2^{(1)}$
- $z_3^{(1)} = x_1^{(0)} * w_{13}^{(1)} + x_2^{(0)} * w_{23}^{(1)} + b_3^{(1)}$
- $a_1^{(1)} = g(z_1^{(1)})$
- $a_2^{(1)} = g(z_2^{(1)})$
- $a_3^{(1)} = g(z_3^{(1)})$

Tổng quát hóa:

$$\begin{aligned} z^{(1)} &= \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \\ z_3^{(1)} \end{bmatrix} = \begin{bmatrix} x_1^{(0)} * w_{11}^{(1)} + x_2^{(0)} * w_{21}^{(1)} + b_1^{(1)} \\ x_1^{(0)} * w_{12}^{(1)} + x_2^{(0)} * w_{22}^{(1)} + b_2^{(1)} \\ x_1^{(0)} * w_{13}^{(1)} + x_2^{(0)} * w_{23}^{(1)} + b_3^{(1)} \end{bmatrix} \\ &= \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} & w_{13}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} & w_{23}^{(1)} \end{bmatrix}^T * \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \end{bmatrix} \\ &= W^{(1)T} * x^{(0)} + b^{(1)} \end{aligned}$$

$$a^{(1)} = g(z^{(1)})$$

Tại Hidden layer 2:

- $z_1^{(2)} = a_1^{(1)} * w_{11}^{(2)} + a_2^{(1)} * w_{21}^{(2)} + a_3^{(1)} * w_{31}^{(2)} + b_1^{(2)}$

- $z_2^{(2)} = a_1^{(1)} * w_{12}^{(2)} + a_2^{(1)} * w_{22}^{(2)} + a_3^{(1)} * w_{32}^{(2)} + b_2^{(2)}$
- $z_3^{(2)} = a_1^{(1)} * w_{13}^{(2)} + a_2^{(1)} * w_{23}^{(2)} + a_3^{(1)} * w_{33}^{(2)} + b_3^{(2)}$
- $a_1^{(2)} = g(z_1^{(2)})$
- $a_2^{(2)} = g(z_2^{(2)})$
- $a_3^{(2)} = g(z_3^{(2)})$

Tương tự như ở Hidden layer 1:

$$z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix} = W^{(2)T} * a^{(1)} + b^{(2)}$$

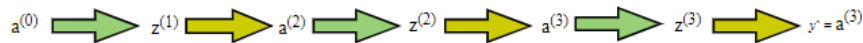
$$a^{(2)} = g(z^{(2)})$$

Tại Output layer:

$$z_1^{(3)} = a_1^{(2)} * w_{11}^{(3)} + a_2^{(2)} * w_{21}^{(3)} + a_3^{(2)} * w_{31}^{(3)} + b_1^{(3)}$$

$$\hat{y} = a_1^{(3)} = g(z_1^{(3)})$$

Vậy quá trình Feedforward như hình dưới đây:



Hình 3: Quá trình Feedforward

3.1.3 Cost và Loss Function

3.1.3.1. Cost function

Là một khái niệm quan trọng trong học máy, đặc biệt là trong mạng neural network. Nó đo lường mức độ sai lệch giữa giá trị dự đoán của mô hình và giá trị thực tế. Việc chọn đúng hàm mất mát là rất quan trọng để đảm bảo mô hình học được tốt nhất.

Với $a = \hat{y}$ là kết quả dự đoán mà mô hình neural network đã cho kết quả, y là kết quả thực tế, hàm mất mát (cost function) cho ta thấy sự khác nhau giữa chúng sau mỗi lần học.

Một số cost function thường được sử dụng:

- **Linear cost function:** Hàm mất mát tuyến tính đơn giản tính toán hiệu số giữa giá trị dự đoán và giá trị thực tế.

$$L(\hat{y}(x), y) = \frac{1}{2}(\hat{y} - y)^2$$

⇒ Hàm này dễ hiểu và dễ triển khai nhưng sai số lớn và ít được sử dụng trong các bài toán học máy hiện đại.

- **Binary Cross-entropy cost function:** Hàm mất mát nhị phân thường được sử dụng cho các bài toán phân loại nhị phân (có hai lớp). Nó đo lường sự khác biệt giữa phân phối xác suất dự đoán và phân phối xác suất thực tế.

$$L(\hat{y}(x), y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

*Nếu $y = 1$:

$$L(\hat{y}(x), y) = -\log(\hat{y})$$

Hàm L giảm dần khi $\hat{y}$ đi từ 0 đến 1. Khi đó model dự đoán $\hat{y}$ gần 1 thì giá trị dự đoán rất gần với giá trị thật. Khi model dự đoán $\hat{y}$ gần 0 thì giá trị dự đoán rất xa so với giá trị thật nên L rất lớn.

*Nếu $y = 0$:

$$L(\hat{y}(x), y) = -\log(1 - \hat{y})$$

Hàm L tăng dần khi $\hat{y}$ đi từ 0 đến 1. Khi đó model dự đoán $\hat{y}$ gần 0 thì giá trị dự đoán rất gần với giá trị thật. Khi model dự đoán $\hat{y}$ gần 1 thì giá trị dự đoán rất xa so với giá trị thật nên L rất lớn.

$\Rightarrow$ Hàm này phù hợp với mô hình output là xác suất (như Sigmoid), đánh giá sai số một cách chính xác hơn.

- **Categorical Cross-Entropy cost function:** Là một hàm mất mát thường được sử dụng trong các bài toán phân loại đa lớp. Nó đo lường sự khác biệt giữa phân phối xác suất dự đoán của mô hình và phân phối xác suất thực tế của nhãn. Nói cách khác, nó đánh giá mức độ "không chắc chắn" của mô hình khi đưa ra dự đoán.

- Giả sử chúng ta có một bài toán phân loại với K lớp:

$$L = - \sum_{k=0}^{K-1} y_k \log(\hat{y}_k)$$

- Trong đó, $y_k \log(\hat{y}_k)$ chỉ tính toán cho các lớp mà nhãn thực tế là 1 (tức là lớp đúng). Giá trị của $\log(\hat{y}_k)$ sẽ càng nhỏ khi $\hat{y}_k$ càng gần 0. Điều này có nghĩa là nếu mô hình dự đoán sai lớp, giá trị của hàm mất mát sẽ lớn, và ngược lại. - Dấu trừ phía trước đảm bảo rằng giá trị của hàm mất mát luôn dương. $\Rightarrow$ Hàm này được thiết kế đặc biệt để đánh giá hiệu suất của các mô hình phân loại đa lớp. Nó so sánh trực tiếp phân phối xác suất dự đoán và phân phối xác suất thực tế, cho phép đánh giá một cách chính xác mức độ tin cậy của dự đoán. Hàm này có khả năng phân biệt tốt giữa các lớp.

3.1.3.2. Lost function

Là cost trên toàn bộ dữ liệu, được định nghĩa như sau:

$$J(w, b) = \frac{1}{N} \sum_{i=1}^N L(\hat{y}(x), y)$$

Loss function là trung bình tất cả các cost trên toàn bộ dữ liệu, nó phụ thuộc vào trọng số w và độ lệch bias b .

Hàm L nhỏ khi giá trị model dự đoán gần với giá trị thật và rất lớn khi model dự đoán sai hay nói cách khác L càng nhỏ thì model dự đoán càng gần với giá trị thật. Mục tiêu của việc training data là làm giảm thiểu loss đến mức tối đa.

$\Rightarrow$ Bài toán trở thành tìm giá trị nhỏ nhất của hàm L .

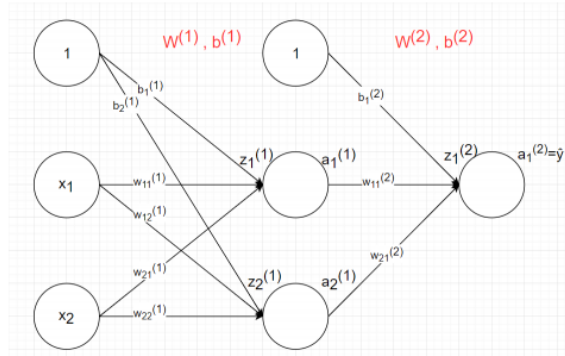
Có thể nghĩ ngay đến thuật toán gradient descent, trong đó nhiệm vụ quan trọng nhất là tìm đạo hàm của các hệ số đối với hàm mất mát (loss function). Việc tính đạo hàm của các hệ số trong mạng nơ-ron được thực hiện thông qua thuật toán backpropagation, sẽ được giới thiệu trong phần tiếp theo.

3.1.4 Backpropagation, Gradient Descent và Learning rate

3.1.4.1. Backpropagation

Backpropagation là thuật toán tối ưu hóa dựa trên gradient descent, được sử dụng rộng rãi để huấn luyện các mạng thần kinh sâu. Bằng cách tính toán đạo hàm của hàm mất mát đối với các trọng số weight và bias b, backpropagation cho phép cập nhật các tham số của mạng theo hướng giảm thiểu sai số. Quá trình truyền ngược sai số này diễn ra từ lớp đầu ra về các lớp ẩn, giúp mạng thần kinh học được các đặc trưng phức tạp từ dữ liệu.

Ta sẽ lấy mạng neural network sau làm ví dụ



Hình 4: Mạng neural điển hình

Feedforward (kí hiệu w và b như hình, có thể xem lại định nghĩa phần 3.1.2.2)

Hidden layer

$$\begin{aligned} \mathbf{z}^{(1)} &= \begin{bmatrix} z_1^{(1)} \\ z_2^{(1)} \end{bmatrix} = \begin{bmatrix} x_1^{(0)} w_{11}^{(1)} + x_2^{(0)} w_{21}^{(1)} + b_1^{(1)} \\ x_1^{(0)} w_{12}^{(1)} + x_2^{(0)} w_{22}^{(1)} + b_1^{(1)} \end{bmatrix} \\ &= \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix}^T \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_1^{(2)} \end{bmatrix} \\ &= \mathbf{W}^{(1)T} \mathbf{x}^{(0)} + \mathbf{b}^{(1)} \end{aligned}$$

$$\Rightarrow a^{(1)} = g\left(\mathbf{z}^{(1)}\right)$$

Output layer:

$$\begin{aligned} \mathbf{z}^{(2)} &= \mathbf{z}_1^{(2)} = \begin{bmatrix} a_1^{(1)} w_{11}^{(2)} + a_2^{(1)} w_{21}^{(2)} + b_1^{(2)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} \\ w_{12}^{(2)} \end{bmatrix}^T \begin{bmatrix} a_1^{(1)} \\ a_2^{(1)} \end{bmatrix} \\ &= \mathbf{W}^{(2)T} \mathbf{a}^{(1)} + \mathbf{b}^{(2)} \end{aligned}$$

$$\Rightarrow \hat{y} = \mathbf{a}^{(2)} = g\left(\mathbf{z}^{(2)}\right)$$

Loss function:

Ở đây, ta dùng hàm cross-entropy cho mỗi điểm $(x[i], y_i)$, gọi hàm loss function:

$$L(x[i], y_i) = -(y_i \log(\hat{y}) + (1 - y_i) \log(1 - \hat{y}))$$

Hàm loss function trên toàn bộ dữ liệu:

$$J(\mathbf{w}, \mathbf{b}) = -\frac{1}{N} \sum_{i=1}^N (y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i))$$

Ta cần tìm model phù hợp nhất với dữ liệu, tương ứng với việc tìm tham số $\mathbf{w}, \mathbf{b}$ để cực tiểu hóa hàm J . Giờ cần một thuật toán để tìm giá trị nhỏ nhất của hàm $J(\mathbf{w}, \mathbf{b})$. Đó chính là thuật toán Gradient Descent.

3.1.4.2. Gradient Descent

Gradient Descent:

Gradient Descent là thuật toán tìm giá trị nhỏ nhất của hàm số dựa trên đạo hàm. Thuật toán hoạt động như sau:

Thuật toán:

- Khởi tạo giá trị $x = x_0$
- Gán $x = x - \text{learning_rate} \cdot f'(x)$
- Tính lại $f(x)$, nếu $f(x)$ đã đủ nhỏ thì dừng lại, ngược lại quay lại bước 2.

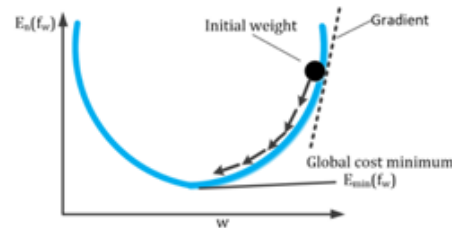
Dựa vào thuật toán trên, chúng ta sẽ điều chỉnh các tham số trong mô hình. Để làm điều này, ta cần tính đạo hàm riêng của hàm mất mát J theo từng tham số:

$$\delta w = \frac{\partial J}{\partial w} \quad \text{và} \quad \delta b = \frac{\partial J}{\partial b}$$

Sau khi tính toán các đạo hàm, chúng ta sẽ cập nhật w và b theo các phần nhỏ dựa trên giá trị của đạo hàm:

$$w = w - \alpha \cdot \delta w$$

$$b = b - \alpha \cdot \delta b$$



Hình 5: Hình cập nhật trọng số w

Trong đó, α là learning rate, một tham số điều chỉnh tốc độ học của mô hình.

Ta sẽ tiến hành bước đạo hàm:

$$L(x[i], y_i) = -(y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y}))$$

với $a^{(2)} = \hat{y}$.

$$\Rightarrow \frac{\partial L}{\partial a^{(i)}} = -\left(\frac{y}{a^{(i)}} - \frac{1-y}{1-a^{(i)}}\right) = -\frac{y(1-a^{(i)}) - (1-y)a^{(i)}}{a^{(i)}(1-a^{(i)})} = \frac{a^{(i)} - y}{a^{(i)}(1-a^{(i)})}$$

Hàm kích hoạt (activation function) ta sẽ chọn hàm Sigmoid:

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\Rightarrow \sigma'(z) = \frac{\partial a}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}}\right) = a(1 - a)$$

Tính đạo hàm L với $w^{(2)}, b^{(2)}$:

$$\frac{\partial L}{\partial b^{(2)}} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial b^{(2)}}$$

$$\frac{\partial L}{\partial w^{(2)}} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial w^{(2)}}$$

Mà:

$$\begin{aligned}\frac{\partial a^{(2)}}{\partial b^{(2)}} &= \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} \\ \frac{\partial a^{(2)}}{\partial w^{(2)}} &= \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w^{(2)}}\end{aligned}$$

Suy ra:

$$\frac{\partial L}{\partial b^{(2)}} = \left(\frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = \frac{a^{(2)} - y}{a^{(2)}(1 - a^{(2)})} \cdot a^{(2)}(1 - a^{(2)}) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = (a^{(2)} - y) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = \delta z^{(2)} \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}}$$

Tương tự,

$$\frac{\partial L}{\partial w^{(2)}} = \left(\frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial w^{(2)}} = \delta z^{(2)} \cdot \frac{\partial z^{(2)}}{\partial w^{(2)}}$$

Tiếp tục với:

$$\begin{aligned}z_1^{(2)} &= a_1^{(1)} \cdot w_{11}^{(2)} + a_2^{(1)} \cdot w_{21}^{(2)} + b_1^{(2)} \\ a_1^{(1)} &= \frac{1}{1 + e^{-z_1^{(1)}}}, \quad a_2^{(1)} = \frac{1}{1 + e^{-z_2^{(1)}}}\end{aligned}$$

Lần lượt tính các đạo hàm riêng:

$$\begin{aligned}\frac{\partial z_1^{(2)}}{\partial w_{11}^{(2)}} &= a_1^{(1)}, \quad \frac{\partial z_1^{(2)}}{\partial w_{21}^{(2)}} = a_2^{(1)}, \quad \frac{\partial z_1^{(2)}}{\partial b_1^{(2)}} = 1 \\ \implies \frac{\partial z^{(2)}}{\partial W^{(2)}} &= a_i^{(1)}, \quad \frac{\partial z^{(2)}}{\partial b^{(2)}} = 1\end{aligned}$$

Suy ra:

$$\frac{\partial L}{\partial b^{(2)}} = \left(\frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = \frac{a^{(2)} - y}{a^{(2)}(1 - a^{(2)})} \cdot a^{(2)}(1 - a^{(2)}) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = (a^{(2)} - y) \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = \delta z^{(2)} \cdot \frac{\partial z^{(2)}}{\partial b^{(2)}} = (a^{(2)} - y)$$

$$\frac{\partial L}{\partial z^{(2)}} = (a^{(2)} - y)$$

Tương tự:

$$\frac{\partial L}{\partial w^{(2)}} = \left(\frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial w^{(2)}} = (a^{(2)} - y) \cdot a_i^{(1)}$$

Lại có:

$$\frac{\partial L}{\partial a_1^{(1)}} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial a_1^{(1)}}$$

Với:

$$\begin{aligned}\frac{\partial a^{(2)}}{\partial a_1^{(1)}} &= \frac{\partial}{\partial a_1^{(1)}} \left(\frac{1}{1 + e^{-(a_1^{(1)} \cdot w_{11}^{(2)} + a_2^{(1)} \cdot w_{21}^{(2)} + b_1^{(2)})}} \right) \\ &= \frac{w_{11}^{(2)} \cdot e^{-(a_1^{(1)} \cdot w_{11}^{(2)} + a_2^{(1)} \cdot w_{21}^{(2)} + b_1^{(2)})}}{(1 + e^{-(a_1^{(1)} \cdot w_{11}^{(2)} + a_2^{(1)} \cdot w_{21}^{(2)} + b_1^{(2)})})^2} = w_{11}^{(2)} \cdot a^{(2)}(1 - a^{(2)})\end{aligned}$$

Suy ra:

$$\frac{\partial L}{\partial a_1^{(1)}} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial a_1^{(1)}} = \frac{a^{(2)} - y}{a^{(2)}(1 - a^{(2)})} \cdot w_{11}^{(2)} \cdot a^{(2)}(1 - a^{(2)}) = w_{11}^{(2)} \cdot (a^{(2)} - y)$$

Tương tự:

$$\frac{\partial L}{\partial a_2^{(1)}} = \frac{\partial L}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial a_2^{(1)}} = \frac{a^{(2)} - y}{a^{(2)}(1 - a^{(2)})} \cdot w_{21}^{(2)} \cdot a^{(2)}(1 - a^{(2)}) = w_{21}^{(2)} \cdot (a^{(2)} - y)$$

Sau quá trình đạo hàm, ta có các đạo hàm của hàm mất mát L với các tham số $W^{(2)}, b^{(2)}$:

- Đạo hàm theo biến $w_{11}^{(2)}$:

$$\frac{\partial L}{\partial w_{11}^{(2)}} = a_1^{(1)} \cdot (\hat{y} - y)$$

- Đạo hàm theo biến $w_{21}^{(2)}$:

$$\frac{\partial L}{\partial w_{21}^{(2)}} = a_2^{(1)} \cdot (\hat{y} - y)$$

- Đạo hàm theo biến $b_1^{(2)}$:

$$\frac{\partial L}{\partial b_1^{(2)}} = \hat{y} - y$$

- Đạo hàm theo biến $a_1^{(1)}$:

$$\frac{\partial L}{\partial a_1^{(1)}} = w_{11}^{(2)} \cdot (\hat{y} - y)$$

- Đạo hàm theo biến $a_2^{(1)}$:

$$\frac{\partial L}{\partial a_2^{(1)}} = w_{21}^{(2)} \cdot (\hat{y} - y)$$

Tính đạo hàm L với $w^{(1)}, b^{(1)}$ với $\hat{y} = a^{(2)}$:

Do $a_1^{(1)} = \sigma(x_1 \cdot w_{11}^{(2)} + x_2 \cdot w_{21}^{(2)} + b_1^{(1)})$

Ta có:

$$\frac{\partial a_1^{(1)}}{\partial b_1^{(1)}} = \sigma'(x_1 \cdot w_{11}^{(2)} + x_2 \cdot w_{21}^{(2)} + b_1^{(1)}) = a_1^{(1)}(1 - a_1^{(1)})$$

Suy ra:

$$\frac{\partial L}{\partial b_1^{(1)}} = \frac{\partial L}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial b_1^{(1)}} = w_{11}^{(2)} \cdot (\hat{y} - y) \cdot a_1^{(1)}(1 - a_1^{(1)})$$

Tương tự cho các biến còn lại:

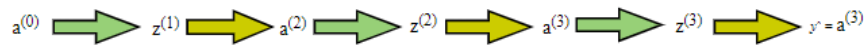
- $\frac{\partial L}{\partial b_2^{(1)}} = \frac{\partial L}{\partial a_2^{(1)}} \cdot \frac{\partial a_2^{(1)}}{\partial b_2^{(1)}} = w_{21}^{(2)} \cdot (\hat{y} - y) \cdot a_2^{(1)}(1 - a_2^{(1)})$
- $\frac{\partial L}{\partial w_{11}^{(1)}} = \frac{\partial L}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{11}^{(1)}} = w_{11}^{(2)} \cdot (\hat{y} - y) \cdot (x_1 \cdot a_1^{(1)}(1 - a_1^{(1)}))$
- $\frac{\partial L}{\partial w_{12}^{(1)}} = \frac{\partial L}{\partial a_2^{(1)}} \cdot \frac{\partial a_2^{(1)}}{\partial w_{12}^{(1)}} = w_{21}^{(2)} \cdot (\hat{y} - y) \cdot (x_1 \cdot a_2^{(1)}(1 - a_2^{(1)}))$
- $\frac{\partial L}{\partial w_{21}^{(1)}} = \frac{\partial L}{\partial a_1^{(1)}} \cdot \frac{\partial a_1^{(1)}}{\partial w_{21}^{(1)}} = w_{11}^{(2)} \cdot (\hat{y} - y) \cdot (x_2 \cdot a_1^{(1)}(1 - a_1^{(1)}))$
- $\frac{\partial L}{\partial w_{22}^{(1)}} = \frac{\partial L}{\partial a_2^{(1)}} \cdot \frac{\partial a_2^{(1)}}{\partial w_{22}^{(1)}} = w_{21}^{(2)} \cdot (\hat{y} - y) \cdot (x_2 \cdot a_2^{(1)}(1 - a_2^{(1)}))$

Vậy là đã tính xong hết đạo hàm của hàm mất mát với các hệ số W và bias b , giờ có thể áp dụng gradient descent để giải bài toán.

Bây giờ, ta áp dụng Gradient descent để tối ưu w và b :

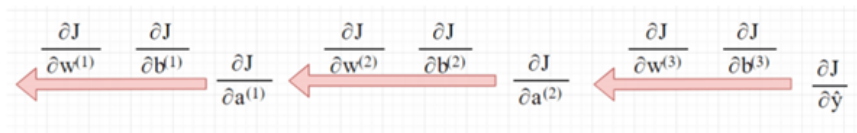
- $w_{11}^{(2)} \leftarrow w_{11}^{(2)} - \alpha \cdot \delta w_{11}^{(2)}$
- $w_{21}^{(2)} \leftarrow w_{21}^{(2)} - \alpha \cdot \delta w_{21}^{(2)}$
- $w_{11}^{(1)} \leftarrow w_{11}^{(1)} - \alpha \cdot \delta w_{11}^{(1)}$
- $w_{12}^{(1)} \leftarrow w_{12}^{(1)} - \alpha \cdot \delta w_{12}^{(1)}$
- $w_{21}^{(1)} \leftarrow w_{21}^{(1)} - \alpha \cdot \delta w_{21}^{(1)}$
- $w_{22}^{(1)} \leftarrow w_{22}^{(1)} - \alpha \cdot \delta w_{22}^{(1)}$
- $b_1^{(2)} \leftarrow b_1^{(2)} - \alpha \cdot \delta b_1^{(2)}$
- $b_1^{(1)} \leftarrow b_1^{(1)} - \alpha \cdot \delta b_1^{(1)}$
- $b_2^{(1)} \leftarrow b_2^{(1)} - \alpha \cdot \delta b_2^{(1)}$

Nếu ở bài trước quá trình feedforward



Hình 6: Quá trình Feedforward

Thì ở bài này quá trình tính đạo hàm ngược lại từ output



Hình 7: Quá trình backpropagation

⇒ Quá trình này gọi là *backpropagation*.

3.1.4.3. Learning rate

Learning rate (tốc độ học) là một siêu tham số quan trọng trong quá trình huấn luyện mô hình học sâu. Nó quyết định bước nhảy của các tham số (trọng số và bias) trong mỗi lần cập nhật. Việc lựa chọn learning rate phù hợp sẽ ảnh hưởng trực tiếp đến hiệu quả và tốc độ hội tụ của mô hình.

Việc chọn hệ số Learning rate rất quan trọng và có 3 trường hợp:

- Nếu learning rate nhỏ: mỗi lần hàm số giảm rất ít nên cần rất nhiều lần thực hiện bước 2 để hàm số đạt giá trị nhỏ nhất.
- Nếu learning rate hợp lý: sau một số lần lặp bước 2 vừa phải thì hàm sẽ đạt giá trị đủ nhỏ.
- Nếu learning rate quá lớn: sẽ gây hiện tượng overshoot (như trong hình dưới) và không bao giờ đạt được giá trị nhỏ nhất của hàm.

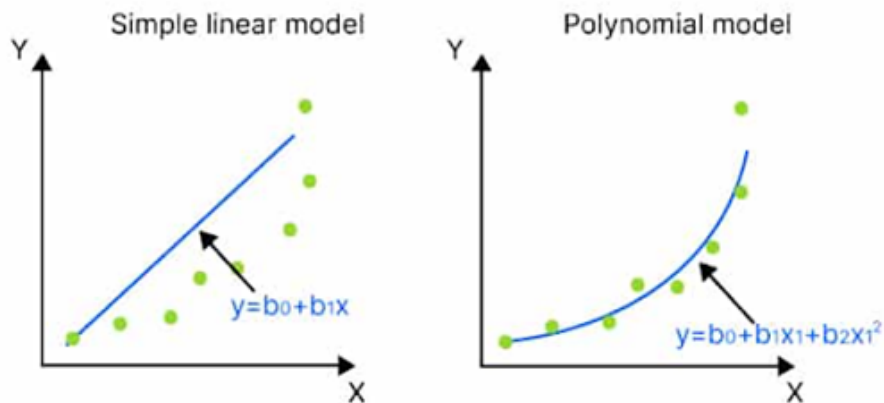


Hình 8: 3 trường hợp của learning rate

3.1.5 Một số Activation Function

3.1.5.1. Vì sao cần có hàm kích hoạt (activation function)?

Như các bạn đã biết thì một bài toán đơn giản trong học máy là linear regression ở đó sử dụng một đường thẳng hoặc một siêu phẳng để biểu diễn mô hình nhưng thực tế thì data sẽ có phân bố phức tạp và sử dụng một hàm tuyến tính là không đủ mạnh để có thể biểu diễn



Như hình trên bạn có thể thấy rằng việc sử dụng một hàm tuyến tính không cho một biểu diễn tốt như hàm bậc hai. Và đối với những vấn đề lớp khác như xử lý ngôn ngữ tự nhiên, computer vision thì việc sử dụng một hàm tuyến tính để mô hình hóa là gần như không thể và ta cần phải mô hình hóa bằng sự phi tuyến.

Giữ các giá trị output trong khoảng nhất định. Giả sử chúng ta không sử dụng activation function và với một model với hàng triệu tham số thì kết quả của phép nhân tuyến tính từ phương trình sẽ có thể là một giá trị rất lớn (dương vô cùng) hoặc rất bé (âm vô cùng) và có thể gây ra những vấn đề về mặt tính toán và mạng rất khó để có thể hội tụ. Việc sử dụng activation function có thể giới hạn đầu ra ở một khoảng giá trị nào đó, ví dụ như hàm sigmoid, softmax giới hạn giá trị đầu ra trong khoảng (0, 1) cho dù kết quả của phép nhân tuyến tính là bao nhiêu đi chăng nữa.

3.1.5.2. Một số hàm activation thường dùng

*Sigmoid

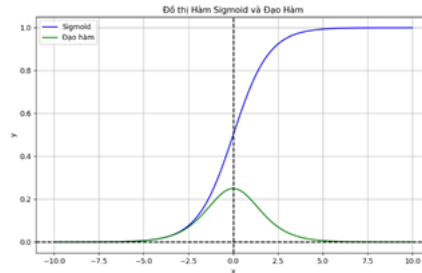
Hàm sigmoid là một hàm liên tục, khả vi và bị chặn trong khoảng $(0, 1)$. Công thức của hàm sigmoid như sau:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Miền xác định: $(-\infty; +\infty)$

Miền giá trị: $(0; 1)$

Đạo hàm: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$



Hình 9: Hình minh họa hàm Sigmoid

Đây là một hàm được ưa chuộng trong quá khứ với đặc điểm có tính đạo hàm tại mọi điểm. Tuy nhiên, hiện nay nó ít được sử dụng hơn do một số lí do như giá trị đạo hàm của nó bị chặn trong khoảng $(0, 0.25)$, do đó nó dễ gây hiện tượng vanishing gradient. Ngoài ra, việc sử dụng hàm mũ khiến cho việc tính toán trở nên lâu hơn. Nói chung, hàm sigmoid thường được sử dụng trong các bài toán hồi quy logistic hoặc trong các cơ chế attention như CBAM, SEblock,...

*Tanh

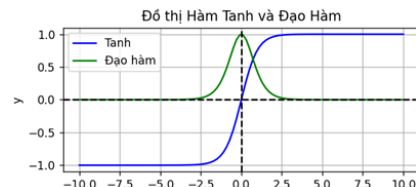
Hàm tanh có hình dáng tương tự như hàm sigmoid nhưng có tính đối xứng qua gốc tọa độ và cũng có các tính chất tương tự như hàm sigmoid. Công thức của hàm tanh như sau:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Miền xác định: $(-\infty; +\infty)$

Miền giá trị: $(-1; 1)$

Đạo hàm: $\tanh'(x) = 1 - \tanh^2(x)$



Hình 10: Hình minh họa hàm Tanh

*ReLU (Rectified Linear Unit)

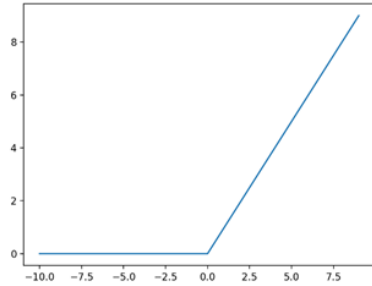
Đây là một hàm activation rất được ưa chuộng sử dụng. Công thức hàm ReLU như sau:

$$\text{ReLU}(x) = \max(0, x)$$

Miền xác định: $(-\infty; +\infty)$

Miền giá trị: $[0; +\infty)$

Đạo hàm: $\text{ReLU}'(x) = \begin{cases} 1 & \text{nếu } x > 0 \\ 0 & \text{nếu } x < 0 \end{cases}$



Hình 11: Hình minh họa hàm ReLU

Ưu điểm của hàm Relu là tính đơn giản của nó và nó đã được chứng minh là giúp tăng tốc quá trình training. Tiếp theo là nó không bị chặn như hàm sigmoid hay Tanh nên nó không phải là nguyên nhân gây ra hiện tượng vanishing gradient. Mặc dù vậy thì tại những điểm có giá trị âm thì giá trị của Relu sẽ bằng 0 (dying relu) và theo lý thuyết nó sẽ không có đạo hàm tại các điểm 0 nhưng thực tế thì người ta thường bổ sung thêm đạo hàm của relu tại 0 bằng 0 và bằng thực nghiệm người ta cũng thấy rằng xác suất để input relu rơi vào điểm 0 là rất nhỏ. Và do nó ko được chặn trên nên cũng có một nhược điểm là gây ra hiện tượng exploding gradient nhưng thường sẽ relu sẽ hoạt động tốt trong thực tế.

*Hàm Leaky ReLU

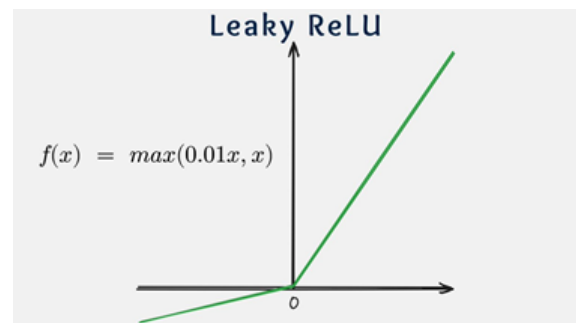
Như đã nói bên trên thì Relu có một nhược điểm là nếu input < 0 thì output của relu sẽ bằng 0 (dying relu) dẫn đến một số node mất ngay lập tức và không học được gì trong quá trình training cả. Do vậy leakyRelu đã khắc phục nhược điểm trên bằng cách sử dụng một siêu tham số alpha. Công thức hàm leakyRelu như sau:

$$\text{Leaky ReLU}(x) = \begin{cases} x & \text{nếu } x > 0 \\ \alpha x & \text{nếu } x \leq 0 \end{cases}$$

Miền xác định: $(-\infty; +\infty)$

Miền giá trị: $(-\infty; +\infty)$

$$\text{Đạo hàm: Leaky ReLU}'(x) = \begin{cases} 1 & \text{nếu } x > 0 \\ \alpha & \text{nếu } x < 0 \end{cases}$$



Hình 12: Hình minh họa hàm Leaky ReLU

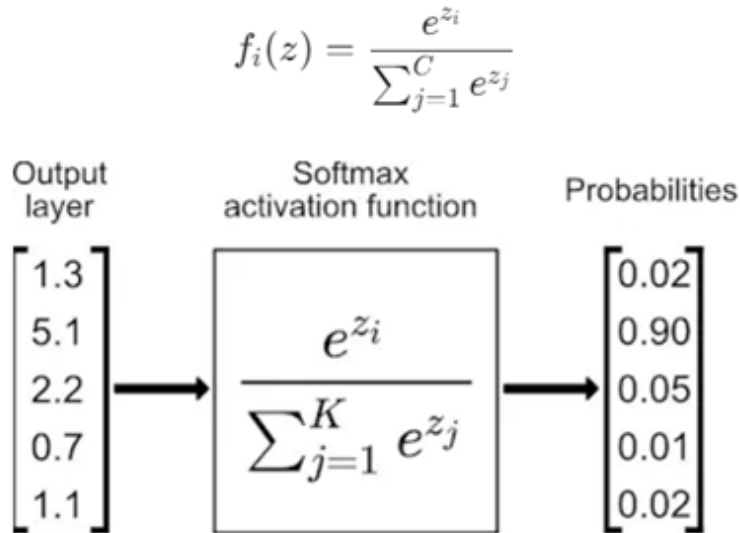
Từ công thức trên ta thấy rằng alpha bằng 1 thì nó trở thành hàm kích hoạt tuyến tính và như đã thảo luận phía trên thì hàm kích hoạt tuyến tính sẽ thường không được sử dụng. Mặc định alpha thường bằng 0.01 nhưng bạn hoàn toàn có thể đặt cho alpha những giá trị mà bạn muốn.

*Softmax

Softmax Đây là một hàm activation thường được sử dụng ở layer cuối cùng của bài classification. Ở đó đầu ra sẽ là xác suất dự đoán rơi vào các class. Công thức hàm softmax như sau:

Với các bài toán classification (phân loại) thì nếu có 2 lớp thì hàm activation ở output layer là hàm sigmoid và hàm loss function là binary_crossentropy, còn nhiều hơn 2 lớp thì hàm activation ở output layer là hàm softmax với loss function là hàm categorical_crossentropy

⇒ Output layer có 10 nodes và activation là softmax function



Hình 13: Hình ảnh minh họa hàm Softmax

3.2 Convolutional Neural Network(CNN)

3.2.1 Xử lý ảnh trong máy tính

Xử lý ảnh trong máy tính là một lĩnh vực nghiên cứu phong phú và đa dạng, bao gồm nhiều kỹ thuật và ứng dụng nhằm giải quyết các vấn đề liên quan đến hình ảnh và video. Các kỹ thuật này không chỉ đơn thuần là cải thiện chất lượng hình ảnh mà còn bao gồm việc trích xuất thông tin có giá trị từ hình ảnh để phục vụ cho nhiều ứng dụng khác nhau, từ nhận diện khuôn mặt cho đến phân tích hình ảnh y tế.

Trong thực tế, quá trình xử lý ảnh thường bao gồm các bước như:

- **Tiền xử lý:** Chỉnh hạn như làm mịn ảnh, loại bỏ nhiễu, hoặc điều chỉnh độ sáng. Mục tiêu là chuẩn bị ảnh để dễ dàng phân tích hơn.
- **Phân đoạn ảnh:** Tách rời các phần của ảnh thành các khu vực có ý nghĩa, giúp nhận diện và phân tích các đối tượng trong ảnh.
- **Nhận diện và phân loại:** Sử dụng các thuật toán học máy hoặc học sâu để xác định đối tượng trong ảnh và phân loại chúng vào các nhóm khác nhau.

Với sự phát triển nhanh chóng của công nghệ học sâu, các mô hình như CNN đã trở thành lựa chọn hàng đầu cho xử lý ảnh. CNN cho phép tự động trích xuất đặc trưng từ dữ liệu mà không cần đến các quy tắc thủ công, giúp tăng cường hiệu suất và độ chính xác của hệ thống. Nhờ vào khả năng học từ lượng dữ liệu lớn, CNN đã đạt được thành công trong nhiều lĩnh vực, chẳng hạn như phân tích cảm xúc từ ảnh, phát hiện bệnh trong hình ảnh y tế, và nhận diện đối tượng trong video.

3.2.2 Phép tính chập (Convolution)

Phép tính chập là một trong những phép toán cốt lõi trong CNN, cho phép mô hình học và trích xuất các đặc trưng từ ảnh đầu vào. Cách thức hoạt động của phép chập có thể được mô tả chi tiết hơn như sau:

- **Bộ lọc (Kernel):** Bộ lọc là một ma trận nhỏ (thường có kích thước 3x3, 5x5 hoặc 7x7) chứa các trọng số, được học trong quá trình huấn luyện. Mỗi bộ lọc sẽ nhận diện một loại đặc trưng nhất định, chẳng hạn như cạnh hoặc kết cấu.

- **Quá trình chập:** Bộ lọc được di chuyển qua bề mặt của ảnh, tại mỗi vị trí, nó thực hiện phép toán tích chập giữa các giá trị pixel của ảnh và các giá trị trong bộ lọc. Kết quả là một giá trị duy nhất cho mỗi vị trí, tạo ra bản đồ đặc trưng (feature map).
- **Tham số stride và padding:**
 - **Stride:** Kích thước bước di chuyển của bộ lọc qua ảnh. Stride lớn hơn 1 sẽ giảm kích thước bản đồ đặc trưng.
 - **Padding:** Việc thêm các pixel giả (thường là 0) xung quanh ảnh để giữ kích thước bản đồ đặc trưng không bị giảm quá nhiều. Việc này cũng giúp cải thiện khả năng phát hiện các đặc trưng ở các cạnh của ảnh.
- **Đặc trưng học được:** Qua nhiều lớp chập, CNN có khả năng học được các đặc trưng từ cơ bản đến phức tạp, từ những cạnh đơn giản trong các lớp đầu tiên cho đến các mẫu phức tạp trong các lớp sâu hơn. Sự chuyển đổi này cho phép mô hình hiểu và nhận diện đối tượng một cách chính xác hơn.

3.2.3 Convolutional Neural Network

Mạng Nơ-ron Tích chập (CNN) là một kiến trúc mạng nơ-ron được thiết kế đặc biệt cho việc xử lý dữ liệu có dạng lưới như ảnh. CNN không chỉ là sự kết hợp của các lớp chập mà còn bao gồm nhiều lớp khác nhau, mỗi lớp có chức năng riêng biệt, tạo ra một quy trình học mạnh mẽ và hiệu quả.

3.2.3.1 Convolutional layer

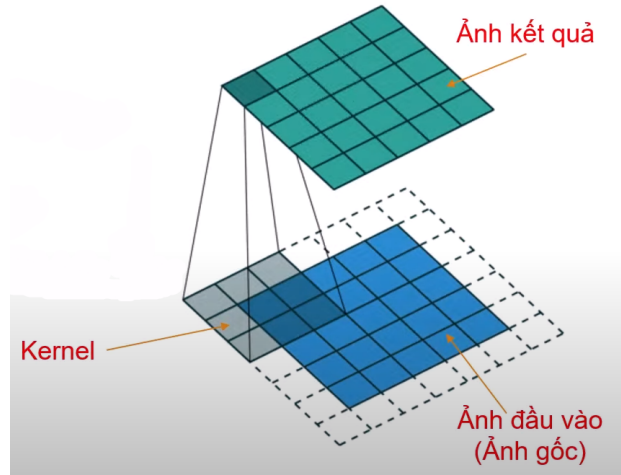
Lớp chập là thành phần chủ chốt của CNN, nơi mà phép chập được thực hiện. Mỗi lớp chập có thể chứa hàng trăm bộ lọc, mỗi bộ lọc được tối ưu hóa để phát hiện các đặc trưng khác nhau. Quá trình học này diễn ra thông qua việc cập nhật trọng số của bộ lọc dựa trên độ chính xác của dự đoán.

- **Số lượng bộ lọc:** Số lượng bộ lọc trong một lớp chập thường tăng dần khi đi sâu vào mạng. Ví dụ, lớp chập đầu tiên có thể chỉ có 32 bộ lọc, trong khi lớp chập cuối cùng có thể có tới 512 bộ lọc.
- **Kết quả:** Kết quả đầu ra của một lớp chập là một hoặc nhiều bản đồ đặc trưng, mỗi bản đồ đại diện cho một loại đặc trưng mà bộ lọc đã phát hiện. Các bản đồ này sẽ được đưa vào lớp pooling hoặc lớp tiếp theo để xử lý thêm.

Chức năng của lớp tích chập: Tạo ra bản đồ kích hoạt/đặc trưng (activation map/feature map) để làm đầu vào cho các lớp.

Cách thức hoạt động:

- Sử dụng kernel, filter và mask là ma trận kích thước: 3x3; 5x5; ...
- Kernel di chuyển trên ảnh và thao tác tích chập.
- Quá trình trên lặp lại đối với tất cả các pixel ảnh gốc.



Hình 14: Minh họa quá trình tích chập

Chú ý:

- Trong giai đoạn train thì giá trị trọng số Kernel được khởi tạo ngẫu nhiên phụ thuộc theo cách thiết kế mạng.
- Để cập nhật lại trọng số thì sử dụng thuật toán lan truyền tiến và lan truyền ngược, giá trị trọng số đã được cập nhật đó là kiến thức mà mạng học được.

Cách tính tích chập với ví dụ sau (với ma trận đầu tiên ứng với một vùng ảnh, ma trận thứ hai là Kernel):

$$\begin{bmatrix} 1 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & -1 \\ -1 & 0 & 1 \end{bmatrix}$$

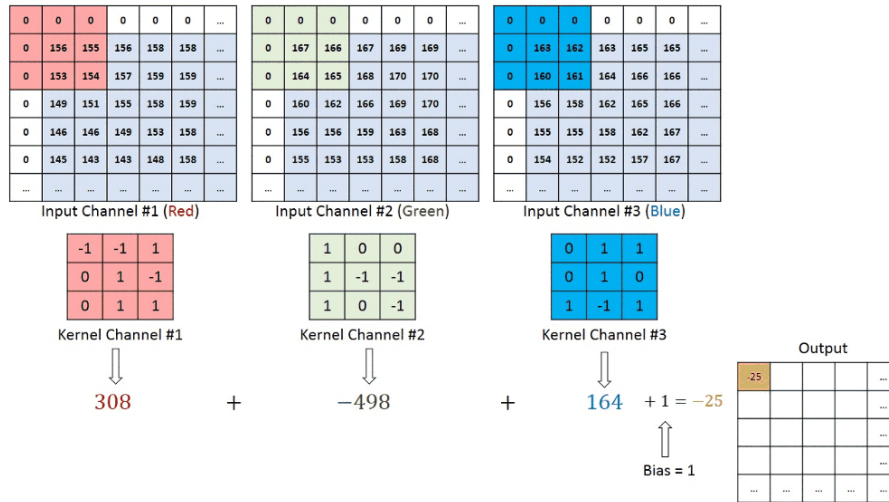
Tích chập $e = 1 * 0 + 0 * 0 + 1 * 0 + 0 * 0 + 0 * 0 + 0 * 1 + 1 * (-1) + 0 * (-1) + 1 * 0 + 0 * 1 = -1$

Quá trình trên lặp lại đối với mỗi điểm ảnh trong ảnh gốc để tạo ra ảnh lọc.

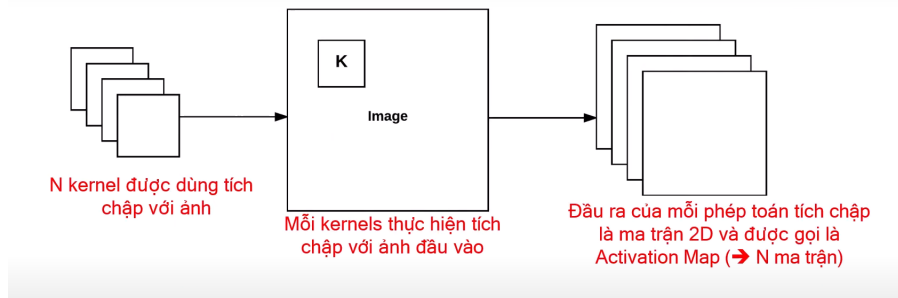
Kích thước ảnh đầu ra sau khi tích chập:

- Trong quá trình di chuyển Kernel, nếu có stride (nhảy bước dài) một giá trị s nào đó thì ảnh đầu ra có kích thước giảm so với ảnh đầu vào.
- Ở các lề của ảnh thiếu các pixel để tạo nên lân cận $\rightarrow$ Thêm p pixel ở lề, gọi là padding (phần đệm).
- Nếu gọi p số pixel thêm vào lề, s là giá trị stride, $m \times n$ là kích thước ảnh đầu vào, $k \times k$ là kích thước kernel, thì kích thước ảnh sau khi tích chập là: $\left(\frac{m-k+2p}{s} + 1\right) \times \left(\frac{n-k+2p}{s} + 1\right)$

Tích chập với ảnh màu: Tích chập cho từng kênh màu rồi lấy tổng.



Hình 15: Minh họa 1 bước của quá trình tích chập với ảnh màu



Hình 16

Trong Hình 16:

- Mỗi Kernel trong N Kernel như trên hình sẽ có ma trận đặc trưng đầu ra riêng.
- Đầu ra của Convolutional layer là các đặc trưng của ảnh đầu vào và gọi là activation map.
- Sau khi có được N activation map thì chúng được xếp chồng lên nhau để tạo thành đầu vào của lớp tiếp theo.

3.2.3.2 Activation layer

Sau mỗi lớp tích chập -> Áp dụng hàm kích hoạt như ReLU, Leaky ReLU, Tanh, sigmoid,...

Kích thước dữ liệu đầu ra của Activation Layer có cùng kích thước dữ liệu đầu vào. Mục đích của lớp Activation là để tăng tính phi tuyến của mô hình

3.2.3.3 Pooling layer

Lớp pooling, hay lớp giảm kích thước, được thiết kế để giảm kích thước của bản đồ đặc trưng và giữ lại các thông tin quan trọng. Quá trình này có tác dụng làm giảm số lượng tham số, do đó làm tăng tốc độ huấn luyện và giảm thiểu hiện tượng overfitting.

- **Max Pooling:** Phương pháp này chọn giá trị lớn nhất từ mỗi vùng con của bản đồ đặc trưng. Việc này giúp giữ lại những đặc trưng nổi bật nhất và làm mượt bản đồ.

- **Average Pooling:** Tính trung bình của các giá trị trong mỗi vùng con, giúp giảm nhiễu và làm cho bản đồ đặc trưng trở nên mượt mà hơn.
- **Chức năng:** Nhờ vào lớp pooling, CNN có khả năng học được các đặc trưng không nhạy cảm với các biến thể nhỏ trong dữ liệu đầu vào, chẳng hạn như sự dịch chuyển hoặc xoay của đối tượng. Điều này giúp tăng cường tính chính xác của mô hình trong các bài toán phân loại thực tế.

Giả sử đầu vào của pooling layer có kích thước $H \times W \times M$, trong đó: - H : Chiều cao của đầu vào - W : Chiều rộng của đầu vào - M : Số lượng kênh (hoặc ma trận) đầu vào

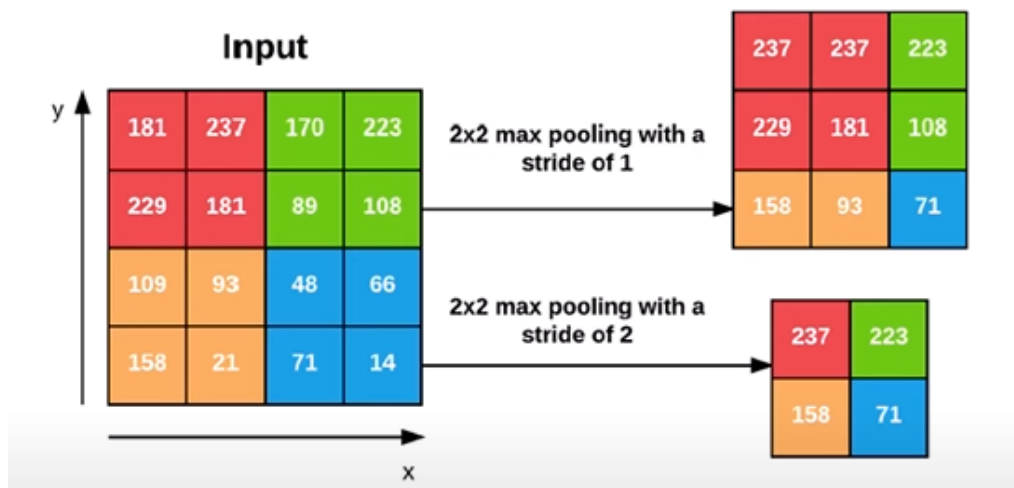
1. Tách đầu vào thành M ma trận kích thước $H \times W$.
2. Với mỗi ma trận, sử dụng kernel có kích thước $P \times P$ để thực hiện một trong hai phép toán sau:
 - Tìm giá trị lớn nhất trong vùng mà kernel được áp dụng (max pooling).
 - Tính giá trị trung bình trong vùng mà kernel được áp dụng (average pooling).
3. Đầu ra sẽ có M ma trận kích thước giảm, kích thước này phụ thuộc vào giá trị stride (S) được sử dụng trong quá trình pooling:

$$H_{out} = \left\lfloor \frac{H - P}{S} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W - P}{S} \right\rfloor + 1$$

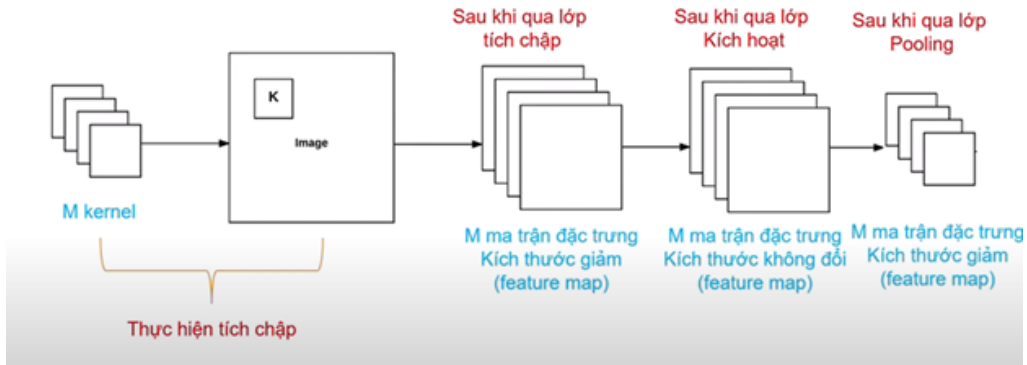
4. Kết quả cuối cùng sẽ là M ma trận có kích thước $H_{out} \times W_{out}$.

Ví dụ



Hình 17: Max Pooling

- Nếu pooling có kích thước 2×2 và stride = 2 thì kích thước giảm một nửa
- * Quá trình qua các lớp đến lớp pooling

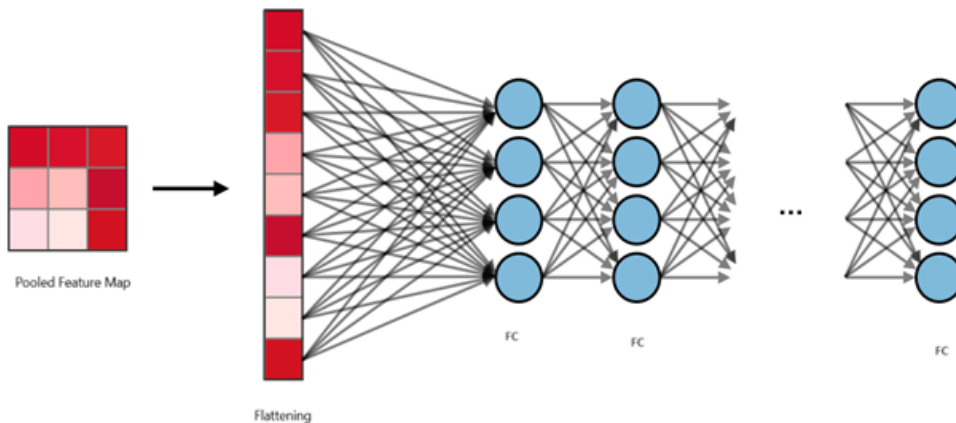


Hình 18: Quá trình qua các lớp đến lớp pooling

3.2.3.4 Fully connected layer

Lớp fully connected là lớp cuối cùng trong kiến trúc CNN. Trong lớp này, tất cả các đầu ra từ các lớp trước đó đều được kết nối với nhau, cho phép mô hình tổng hợp thông tin từ nhiều đặc trưng khác nhau.

- **Kết nối hoàn toàn:** Mỗi nơ-ron trong lớp này nhận thông tin từ tất cả các nơ-ron ở lớp trước đó, từ đó tạo ra một vector xác suất cho mỗi lớp phân loại. Điều này giúp mô hình đưa ra quyết định cuối cùng về lớp nào là phù hợp nhất cho đầu vào.
- **Hàm kích hoạt:** Thường sử dụng hàm kích hoạt softmax để chuyển đổi đầu ra thành xác suất. Điều này cho phép mô hình đưa ra dự đoán cho nhiều lớp cùng một lúc.
- **Quá trình huấn luyện:** Trong quá trình huấn luyện, trọng số của các nơ-ron trong lớp fully connected được cập nhật dựa trên sai số giữa dự đoán của mô hình và nhãn thực tế. Quá trình này diễn ra qua nhiều vòng lặp (epochs), giúp mô hình dần dần cải thiện độ chính xác của mình.



Hình 19: Fully connected

3.2.4 Các metrics trong bài toán phân loại

Khi đánh giá hiệu suất của mô hình phân loại, việc sử dụng các chỉ số (metrics) chính xác là rất quan trọng. Các chỉ số này giúp hiểu rõ hơn về khả năng của mô hình trong việc phân loại đúng các đối tượng. Các thuật ngữ được sử dụng trong Machine Learning để mô tả hiệu suất của mô hình phân loại là: TP (True Positive), TN (True Negative), FP (False Positive), FN (False Negative)

3.2.4.1 Accuracy

Công thức: $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

Ý nghĩa: Accuracy cung cấp cái nhìn tổng quát về độ chính xác của mô hình. Tuy nhiên, trong các bài toán với dữ liệu không cân bằng, độ chính xác có thể không phản ánh đúng hiệu suất, vì mô hình có thể đạt được độ chính xác cao bằng cách chỉ dự đoán lớp chiếm ưu thế.

3.2.4.2 Precision

Công thức: $Precision = \frac{TP}{TP+FP}$

Ý nghĩa: Precision đo lường độ chính xác của các dự đoán dương tính. Đây là chỉ số quan trọng trong các bài toán mà sai số dương tính giả có thể gây ra hậu quả nghiêm trọng, như trong lĩnh vực y tế hoặc an ninh.

3.2.4.3 Recall

Công thức: $Recall = \frac{TP}{TP+FN}$

Ý nghĩa: Recall cho thấy khả năng của mô hình trong việc nhận diện tất cả các trường hợp dương tính. Một mô hình có recall cao cho thấy nó ít bỏ sót các trường hợp quan trọng.

3.2.4.4 F1 Score

Công thức: $F1Score = 2 * \frac{Precision * Recall}{Precision + Recall}$

Ý nghĩa: F1 Score là chỉ số hữu ích khi bạn cần một sự cân bằng giữa Precision và Recall, đặc biệt trong các bài toán phân loại không cân bằng.

3.2.4.5 Confusion Matrix

	Dự đoán Pos	Dự đoán Neg
Thực tế Pos	TP	FN
Thực tế Neg	FP	TN

Ý nghĩa: Confusion Matrix giúp xác định các lớp nào hoạt động tốt và các lớp nào cần cải thiện, từ đó đưa ra các quyết định điều chỉnh mô hình.

3.3 Một số mô hình đề xuất

3.3.1 Mạng VGG 16

VGG16 là mạng convolutional neural network được đề xuất bởi K. Simonyan and A. Zisserman, University of Oxford. Model sau khi train bởi mạng VGG16 đạt độ chính xác 92.7% top-5 test trong dữ liệu ImageNet gồm 14 triệu hình ảnh thuộc 1000 lớp khác nhau. Giờ áp dụng

kiến thức ở trên để phân tích mạng VGG 16.



Hình 20: Mô hình mạng VGG16

- **Convolutional Layers:**

- **Kích thước kernel:** Mạng VGG16 sử dụng các lớp tích chập với kích thước kernel là 3×3 , đủ nhỏ để bắt các đặc trưng không gian chi tiết, đồng thời giảm số lượng tham số cần học.
- **Stride:** Khi không chỉ định, stride mặc định là 1. Kernel di chuyển một bước một lần trên chiều rộng và chiều cao, giúp phát hiện các đặc trưng cục bộ nhỏ.
- **Padding:** Padding thường được đặt là 1 để giữ nguyên kích thước đầu ra so với đầu vào. Với kernel 3×3 và $\text{stride} = 1$, $\text{padding} = 1$ giữ cho chiều rộng và chiều cao của ảnh không thay đổi.
- **Output Depth:** Convolutional layers sử dụng nhiều bộ lọc (kernels). Ví dụ, lớp đầu tiên có 3×3 conv, 64 nghĩa là 64 kernel được áp dụng, tạo ra đầu ra có độ sâu (depth) là 64.

- **Max Pooling Layers:**

- Max pooling với kích thước kernel 2×2 và $\text{stride} = 2$ giúp giảm kích thước chiều rộng và chiều cao của đầu vào xuống một nửa, giữ lại các đặc trưng quan trọng nhất.

- **Thay đổi kích thước của các đặc trưng:**

- Trong các lớp đầu tiên, kích thước chiều rộng và chiều cao của đầu vào lớn, nhưng sau mỗi lớp pooling, chúng giảm đi một nửa.
- Số lượng kernel tăng dần sau mỗi vài lớp convolution từ 64, 128, 256, đến 512 kernel. Kích thước không gian giảm nhưng độ sâu (depth) tăng lên.

- **Flatten và Fully Connected Layers:**

- **Flatten:** Sau nhiều lớp convolution và pooling, dữ liệu đầu ra có cấu trúc ba chiều. Trước khi đưa vào fully connected layers, dữ liệu cần được flatten thành vector một chiều.
- **Fully Connected Layers:** VGG16 có ba fully connected layers, trong đó hai lớp đầu có 4096 units và lớp cuối cùng có 1000 units tương ứng với 1000 lớp của ImageNet.

- **ReLU Activation Function và Softmax:**

- **ReLU:** Hàm kích hoạt ReLU được sử dụng sau mỗi lớp convolution và fully connected, giúp mạng học hiệu quả hơn và tạo ra tính phi tuyến cho mạng.
- **Softmax:** Ở lớp đầu ra cuối cùng, hàm softmax tạo ra xác suất phân bố cho 1000 lớp đầu ra trong ImageNet.

- *Ưu điểm và Nhược điểm của VGG16:

- Ưu điểm:

- * Kiến trúc đơn giản, mạnh mẽ, và có thể tổng quát hóa tốt cho nhiều bài toán thị giác máy tính.
- * Lớp convolution nhỏ (3×3) giúp học đặc trưng cục bộ hiệu quả.

- Nhược điểm:

- * Số lượng tham số lớn (khoảng 138 triệu) gây tốn tài nguyên tính toán và bộ nhớ.
- * Độ sâu lớn khiến thời gian huấn luyện lâu, đặc biệt khi không có GPU mạnh.

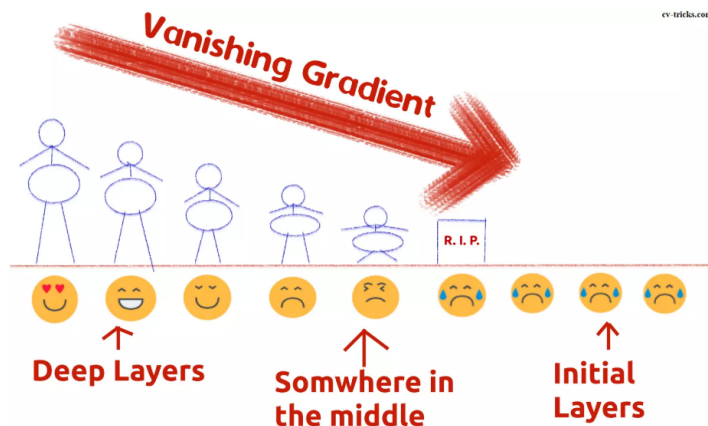
3.3.2 Mạng ResNet

- Giới thiệu

ResNet là một trong những mạng nơ-ron sâu mạnh mẽ nhất đã đạt được kết quả hiệu suất tuyệt vời trong thử thách phân loại **ILSVRC 2015**. ResNet đã đạt được hiệu suất tổng quát tuyệt vời trong các nhiệm vụ nhận dạng khác và giành giải nhất về phát hiện ImageNet, định vị ImageNet, phát hiện **COCO** và phân đoạn **COCO** trong các cuộc thi **ILSVRC** và **COCO 2015**. Có nhiều biến thể của kiến trúc ResNet tức là cùng một khái niệm nhưng có số lớp khác nhau. Chúng ta có **ResNet-18**, **ResNet-34**, **ResNet-50**, **ResNet-101**, **ResNet-110**, **ResNet-152**, **ResNet-164**, **ResNet-1202**, v.v. Tên ResNet theo sau là một số có hai chữ số trở lên chỉ đơn giản ám chỉ kiến trúc ResNet với một số lớp mạng nơ-ron nhất định. Trong bài đăng này, chúng ta sẽ đề cập chi tiết đến **ResNet-50**, một trong những mạng sống động nhất.

- Quá trình ra đời

Từ năm 2013, cộng đồng Học sâu bắt đầu xây dựng các mạng sâu hơn vì chúng có thể đạt được các giá trị độ chính xác cao. Hơn nữa, các mạng sâu hơn có thể biểu diễn các tính năng phức tạp hơn, do đó độ mạnh và hiệu suất của mô hình có thể được tăng lên. Tuy nhiên, việc xếp chồng nhiều lớp hơn không hiệu quả đối với các nhà nghiên cứu. Trong khi đào tạo các mạng sâu hơn, vấn đề suy giảm độ chính xác đã được quan sát thấy. Nói cách khác, việc thêm nhiều lớp vào mạng khiến giá trị độ chính xác bão hòa hoặc đột ngột bắt đầu giảm. Thủ phạm gây ra sự suy giảm độ chính xác là hiệu ứng gradient biến mất (**Vanishing Gradient**) chỉ có thể quan sát thấy trong các mạng sâu hơn.

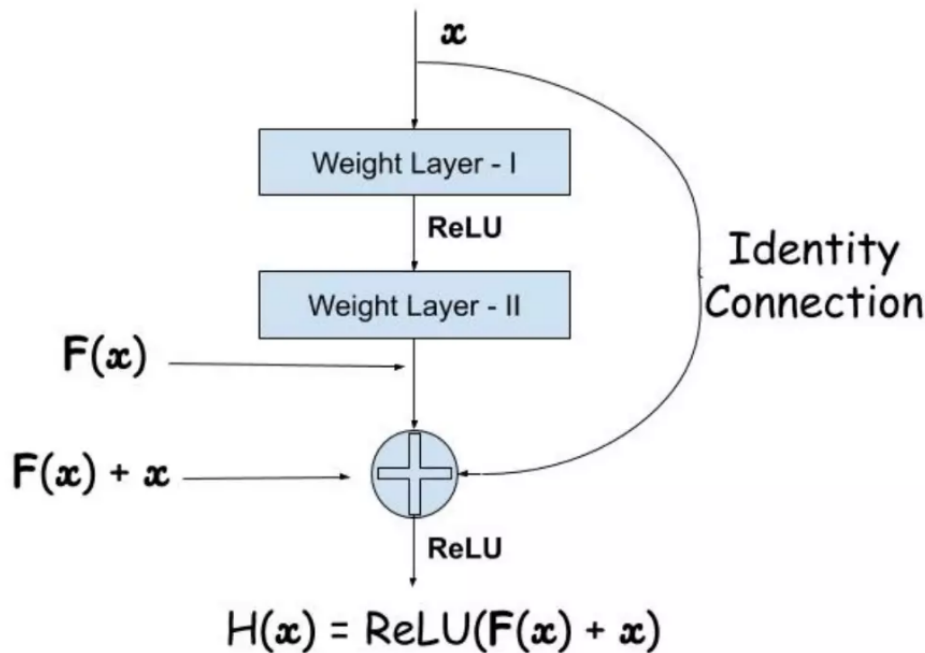


Hình 21: Vanishing Gradient

Trong giai đoạn truyền ngược, lỗi được tính toán và các giá trị gradient được xác định. Các gradient được gửi trở lại các lớp ẩn và các trọng số được cập nhật tương ứng. Quá trình xác định gradient và gửi trở lại lớp ẩn tiếp theo được tiếp tục cho đến khi đạt đến lớp đầu vào. Gradient trở nên nhỏ hơn và nhỏ hơn khi nó đạt đến đáy của mạng. Do đó, các trọng số của các lớp ban đầu sẽ cập nhật rất chậm hoặc vẫn giữ nguyên. Nói cách khác, các lớp ban đầu của mạng sẽ không học hiệu quả. Do đó, đào tạo mạng sâu sẽ không hội tụ và độ chính xác sẽ bắt đầu giảm hoặc bão hòa ở một giá trị cụ thể. Mặc dù vấn đề gradient biến mất đã được giải quyết bằng cách sử dụng khối tạo trọng số được chuẩn hóa, độ chính xác của mạng sâu hơn vẫn không tăng lên.

- Kiến trúc mạng ResNet

ResNet gần giống với các mạng có các lớp tích chập, gộp, kích hoạt và kết nối đầy đủ xếp chồng lên nhau. Cấu trúc duy nhất của mạng đơn giản để biến nó thành mạng dư thừa là kết nối danh tính giữa các lớp. Ảnh chụp màn hình bên dưới hiển thị khối dư thừa được sử dụng trong mạng. Bạn có thể thấy kết nối danh tính là mũi tên cong bắt nguồn từ đầu vào và chìm xuống cuối khối dư thừa.

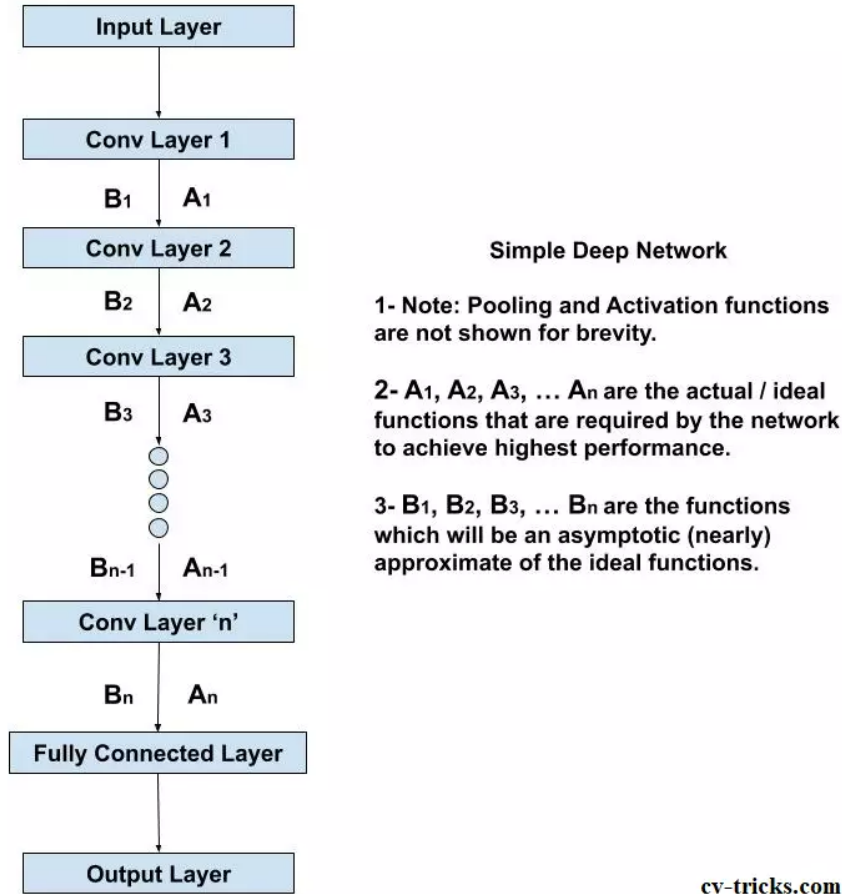


Hình 22: Kiến trúc một khối residual của mạng

Giả sử chúng ta có một mạng sâu gồm nhiều lớp được xếp chồng lên nhau, bao gồm các lớp tích chập, gộp, v.v. Hàm thực tế mà chúng ta đang cố gắng học sau mỗi lớp được biểu diễn bởi $A_i(x)$ trong đó A_i là hàm đầu ra của lớp thứ i cho một đầu vào x đã cho. Các hàm đầu ra sau mỗi lớp có thể được biểu diễn lần lượt là:

$$A_1(x), A_2(x), A_3(x), \dots, A_n(x)$$

Mục tiêu là làm sao để mạng sâu này hoạt động tốt hơn hoặc ít nhất tương đương với các mạng nông hơn, ngay cả khi số lớp tăng lên. Điều này yêu cầu chúng ta phải giải quyết vấn đề độ chính xác bị giảm đột ngột khi tăng số lớp.



Hình 23: Ví dụ Mạng nơ-ron tích chập

Theo cách học này, mạng sẽ trực tiếp cố gắng học các hàm đầu ra này, tức là không có bất kỳ sự hỗ trợ bổ sung nào. Trên thực tế, mạng không thể học các hàm lý tưởng này ($A_1, A_2, A_3, \dots, A_n$). Mạng chỉ có thể học các hàm, chẳng hạn như $B_1, B_2, B_3, \dots, B_n$ gần với $A_1, A_2, A_3, \dots, A_n$ hơn.

Tuy nhiên, mạng sâu giả định của chúng ta tệ hơn nhiều đến nỗi ngay cả nó cũng không thể học $B_1, B_2, B_3, \dots, B_n$ gần với $A_1, A_2, A_3, \dots, A_n$ hơn do hiệu ứng biến mất của gradient và cũng do cách đào tạo không được hỗ trợ.

Hỗ trợ cho việc đào tạo sẽ được cung cấp bằng cách thêm ánh xạ danh tính vào đầu ra còn lại. Trước tiên, chúng ta hãy xem ý nghĩa của ánh xạ danh tính là gì. Nói một cách ngắn gọn, áp dụng ánh xạ danh tính vào đầu vào sẽ cung cấp cho bạn đầu ra giống với đầu vào: $AI = A$ trong đó A là ma trận đầu vào và I là ánh xạ danh tính.

Các mạng truyền thống như AlexNet, VGG-16, v.v. cố gắng học $A_1, A_2, A_3, \dots, A_n$ trực tiếp như thể hiện trong sơ đồ của mạng sâu đơn giản. Trong lần truyền tới, đầu vào (hình ảnh) được truyền tới mạng để lấy đầu ra. Lỗi được tính toán và độ dốc được xác định và lan truyền ngược giúp mạng xấp xỉ các hàm $A_1, A_2, A_3, \dots, A_n$ dưới dạng $B_1, B_2, B_3, \dots, B_n$

Vì sao hàm số dư lại có hiệu quả?

Câu trả lời là: trong quá trình đào tạo mạng lưới dư lượng sâu, trọng tâm chính là học hàm dư lượng, tức là $F(x)$. Nếu mạng lưới bằng cách nào đó học được sự khác biệt ($F(x)$) giữa đầu vào và đầu ra, thì độ chính xác tổng thể có thể được tăng lên.

Nói cách khác, giá trị dư lượng $F(x)$ phải được học theo cách sao cho nó tiến gần đến 0, do đó làm cho ánh xạ danh tính trở nên tối ưu:

$$H(x) = F(x) + x$$

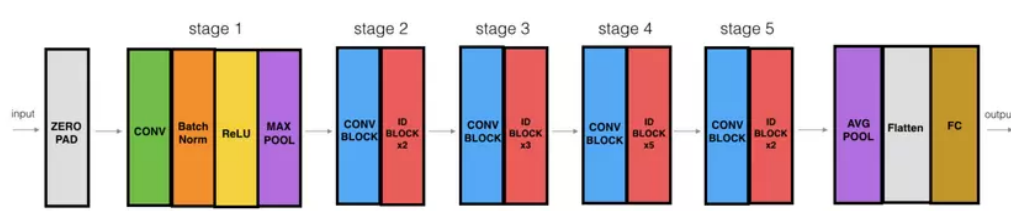
trong đó:

- $H(x)$: là ánh xạ cần học.
- $F(x)$: là hàm dư lượng cần mạng học được.
- x : là đầu vào.

Theo cách này, tất cả các lớp trong mạng sẽ luôn tạo ra các Feature map tối ưu, tức là bản đồ đặc trưng tốt nhất có thể, sau các hoạt động tích chập, gộp và kích hoạt.

Feature map tối ưu chứa tất cả các đặc trưng có liên quan, giúp phân loại hình ảnh một cách hoàn hảo vào lớp thực tế của nó.

- Xây dựng mạng ResNet50



Hình 24: Mô hình mạng

Phân tích chi tiết các giai đoạn trong ResNet-50

*Zero-padding

- Thêm zero vào các cạnh của đầu vào để đảm bảo kích thước đầu vào phù hợp với các lớp tiếp theo.
- Padding được sử dụng với kernel 3×3 .

*Stage 1

- **Tích chập ban đầu (Conv1):**
 - 64 filter, kích thước kernel 7×7 , stride 2×2 .
- **Batch Normalization (BatchNorm):** Giúp ổn định quá trình huấn luyện.
- **MaxPooling:**
 - Kích thước kernel 3×3 , stride 2×2 .

*Stage 2

- **Convolutional Block:**
 - Filter size $64 \times 64 \times 256$, kernel size 3×3 , stride 1.
- **Identity Blocks (x2):**
 - Mỗi Identity Block sử dụng filter size $64 \times 64 \times 256$, kernel size 3×3 .

***Stage 3**

- **Convolutional Block:**
 - Filter size $128 \times 128 \times 512$, kernel size 3×3 , stride 2.
- **Identity Blocks (x3):**
 - Mỗi Identity Block sử dụng filter size $128 \times 128 \times 512$, kernel size 3×3 .

***Stage 4**

- **Convolutional Block:**
 - Filter size $256 \times 256 \times 1024$, kernel size 3×3 , stride 2.
- **Identity Blocks (x5):**
 - Mỗi Identity Block sử dụng filter size $256 \times 256 \times 1024$, kernel size 3×3 .

***Stage 5**

- **Convolutional Block:**
 - Filter size $512 \times 512 \times 2048$, kernel size 3×3 , stride 2.
- **Identity Blocks (x2):**
 - Mỗi Identity Block sử dụng filter size $512 \times 512 \times 2048$, kernel size 3×3 .

***The 2D Average Pooling**

- Kích thước kernel 2×2 , giúp giảm độ sâu của tensor.

***Flatten**

- Chuyển đổi tensor 4D thành vector 1D để chuẩn bị cho lớp Fully Connected.

***Fully Connected (Dense)**

- Sử dụng hàm kích hoạt Softmax để dự đoán xác suất các lớp.

***Ưu điểm và Nhược điểm của ResNet:**

- **Ưu điểm:**
 - Kiến trúc sử dụng residual blocks, giúp giải quyết vấn đề vanishing gradients ở các mạng sâu.
 - Đạt được hiệu suất rất cao trên nhiều bài toán thị giác máy tính mà không gặp phải vấn đề suy giảm hiệu suất khi tăng độ sâu của mạng.
 - Cải thiện tốc độ huấn luyện và khả năng tổng quát hóa so với các mạng CNN truyền thống.
- **Nhược điểm:**
 - Mặc dù giảm thiểu vấn đề vanishing gradients, nhưng độ phức tạp của mạng vẫn có thể dẫn đến thời gian huấn luyện lâu.
 - Sử dụng nhiều phép toán ma trận trong các block residual có thể tốn tài nguyên tính toán và bộ nhớ.

4 Chọn model để hiện thực

4.1 Phân tích dataset

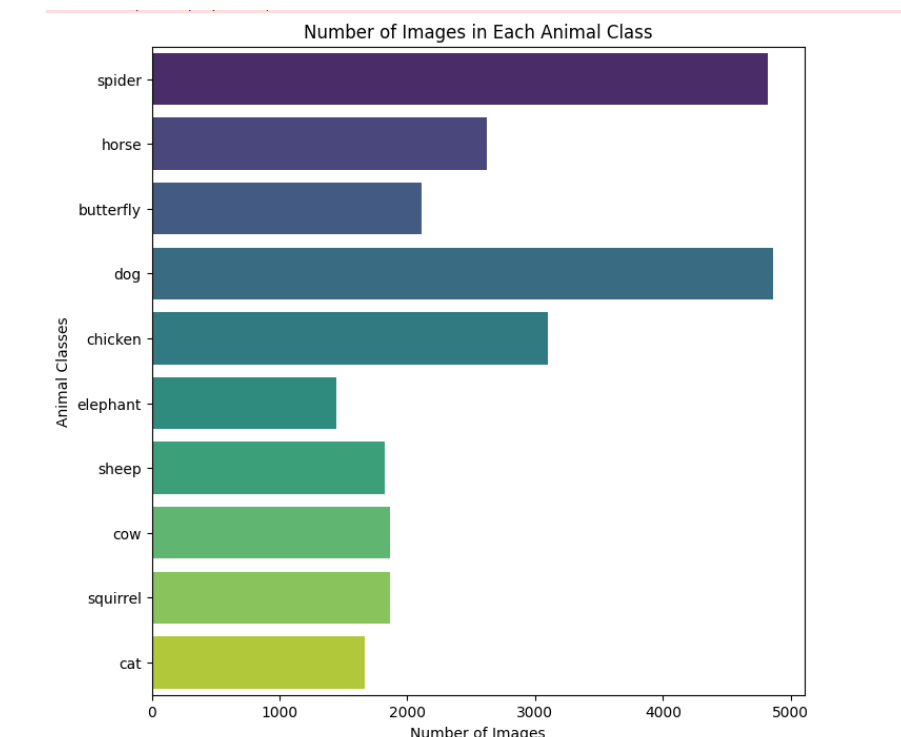
Nhóm đã chọn tập dataset **Animals_10** trên Kaggle gồm 10 class với tổng số ảnh là 26179 ảnh.

- Thông tin các class

Tập dữ liệu **Animals_10** có tổng cộng 26,179 ảnh thuộc 10 lớp khác nhau. Dưới đây là bảng các lớp cùng với chỉ số của chúng:

Chỉ số lớp	Tên lớp	Số lượng ảnh
0	Spider	4821
1	Horse	2623
2	Butterfly	2112
3	Dog	4863
4	Chicken	3098
5	Elephant	1446
6	Sheep	1820
7	Cow	1866
8	Squirrel	1862
9	Cat	1668

Bảng 1: Thông tin các lớp trong dataset



Hình 25: Phân bố số lượng ảnh của từng class

Nhận thấy dữ liệu trong dataset Animals-10 có sự phân bố không đồng đều giữa các lớp, với một số lớp như Butterfly (Class 0) và Cow (Class 3) có số lượng ảnh lớn hơn đáng kể so với các lớp khác như Elephant (Class 5) và Squirrel (Class 9). Sự mất cân bằng này có thể gây ảnh hưởng đến hiệu suất của mô hình, khi các lớp có nhiều ảnh sẽ chiếm ưu thế trong quá trình huấn luyện, trong

khi các lớp ít ảnh có thể bị phân loại sai. Để khắc phục điều này, cần áp dụng các phương pháp cân bằng lớp như oversampling, undersampling. Nhóm sẽ chọn xử lý theo hướng undersampling.

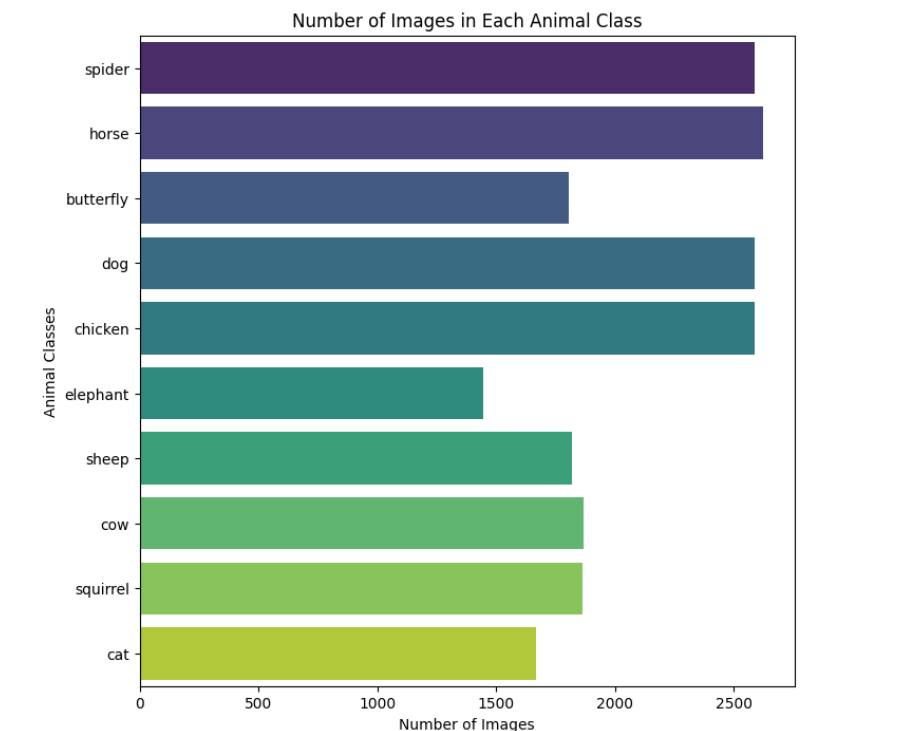
4.2 Xử lý dataset bằng phương pháp Undersampling

Undersampling là một kỹ thuật khác được sử dụng trong các bài toán phân loại với dữ liệu không cân bằng. Thay vì tăng số lượng mẫu cho các lớp ít dữ liệu như trong oversampling, undersampling giảm số lượng mẫu của lớp có số lượng dữ liệu lớn hơn (lớp chiếm ưu thế). Mục đích của phương pháp này là làm cho phân bố của các lớp trở nên cân bằng hơn, giúp mô hình không bị thiên lệch về các lớp có nhiều dữ liệu hơn.

Sau khi áp dụng kỹ thuật trên, thống kê dataset như sau:

Chỉ số lớp	Tên lớp	Số lượng ảnh
0	Spider	2588
1	Horse	2623
2	Butterfly	1807
3	Dog	2588
4	Chicken	2588
5	Elephant	1446
6	Sheep	1820
7	Cow	1866
8	Squirrel	1862
9	Cat	1668

Bảng 2: Thông tin số lượng ảnh trong dataset đã qua xử lý



Hình 26: Phân bố số lượng ảnh của từng class đã xử lý

Một số hình ảnh trong dataset



Hình 27: Một số hình ảnh trong tập data

=> Kết luận

Dựa trên dữ liệu và yêu cầu của bài toán, việc chọn VGG16 thay vì ResNet có thể là sự lựa chọn hợp lý vì một số lý do chính. Thứ nhất, VGG16 có cấu trúc đơn giản và dễ hiểu với các lớp convolutional liên tiếp, giúp dễ dàng triển khai và điều chỉnh, đặc biệt trong các bài toán phân loại ảnh với dữ liệu không quá lớn. Với số lượng tham số lớn nhưng không quá phức tạp, VGG16 có thể đạt được kết quả tốt mà không cần phải sử dụng quá nhiều tài nguyên tính toán. Thứ hai, dataset trong bài toán này không phải quá lớn và có sự phân bố số lượng ảnh trong các lớp không đều, vì vậy VGG16, mặc dù có nhiều tham số, vẫn có thể hoạt động hiệu quả mà không gặp phải vấn đề overfitting như một mô hình phức tạp hơn như ResNet. Cuối cùng, VGG16 đã được chứng minh là hiệu quả trong nhiều bài toán phân loại ảnh cơ bản và có sẵn các mô hình pre-trained trên ImageNet, giúp quá trình huấn luyện và fine-tuning trở nên nhanh chóng và dễ dàng. Do đó, VGG16 là một lựa chọn hợp lý cho bài toán này khi yêu cầu về tính đơn giản và hiệu quả trong việc sử dụng tài nguyên tính toán.

4.3 Tiền xử lý và Tăng cường Dữ liệu (Data Preprocessing and Augmentation)

Trong mô hình này, nhóm sử dụng hai đối tượng `ImageDataGenerator` từ thư viện Keras để thực hiện tiền xử lý và tăng cường dữ liệu cho bộ dữ liệu huấn luyện và kiểm tra.

```
1 train_datagen = ImageDataGenerator(  
2     rescale = 1./255,  
3     rotation_range = 20,  
4     width_shift_range = 0.2,  
5     height_shift_range = 0.2,  
6     shear_range = 0.2,  
7     zoom_range = 0.2,
```

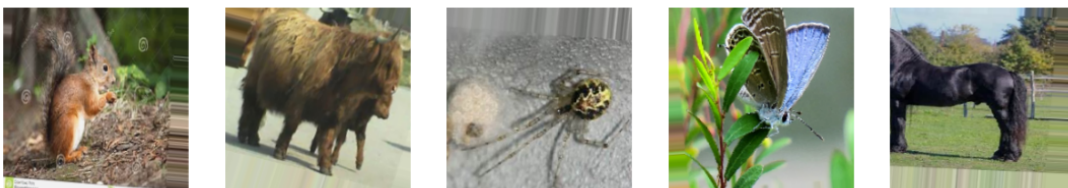
```
8     horizontal_flip = True,
9     fill_mode = 'nearest',
10    brightness_range = [0.8, 1.2],
11    channel_shift_range = 30.0,
12    validation_split = 0.2
13)
14
15 valid_datagen = ImageDataGenerator(
16     rescale = 1./255,
17     validation_split = 0.2
18)
19
20 train_generator = train_datagen.flow_from_directory(
21     '/kaggle/input/cleandataset/dataset',
22     target_size = (224, 224),
23     batch_size = 64,
24     class_mode = 'categorical',
25     subset = 'training'
26)
27
28 validation_generator = valid_datagen.flow_from_directory(
29     '/kaggle/input/cleandataset/dataset',
30     target_size = (224, 224),
31     batch_size = 64,
32     class_mode = 'categorical',
33     subset = 'validation',
34     shuffle = False
35)
```

Found 16689 images belonging to 10 classes.
Found 4167 images belonging to 10 classes.

Hình 28: Kết quả phân chia dataset

```
fig, ax = plt.subplots(nrows=1, ncols=5, figsize=(15,15))

for i in range(5):
    image = next(train_generator)[0][0]
    image = np.squeeze(image)
    ax[i].imshow(image)
    ax[i].axis(False)
```



Hình 29: Data sau khi được Augmentation

* Nhận xét và Đánh giá

Đoạn mã trên áp dụng các phương pháp tiền xử lý và tăng cường dữ liệu để cải thiện khả năng tổng quát của mô hình. Việc sử dụng các kỹ thuật như quay, dịch chuyển, phóng to và đảo chiều hình ảnh giúp mô hình học được các đặc trưng không gian và góc nhìn khác nhau của dữ liệu, điều này giúp giảm thiểu overfitting và cải thiện hiệu suất khi mô hình đối mặt với dữ liệu thực tế. Các tham số như `brightness_range` và `channel_shift_range` giúp mô phỏng các thay đổi về điều kiện ánh sáng, làm cho mô hình trở nên linh hoạt hơn khi xử lý các hình ảnh từ môi trường thực tế.

Việc chia dữ liệu thành các tập huấn luyện và kiểm tra với tỷ lệ 80-20 giúp mô hình có đủ dữ liệu để huấn luyện, đồng thời đảm bảo có một bộ kiểm tra để đánh giá mô hình sau mỗi epoch. Tuy nhiên, cần lưu ý rằng việc sử dụng nhiều phương pháp tăng cường có thể làm tăng thời gian huấn luyện, đặc biệt khi số lượng hình ảnh đầu vào lớn. Do đó, việc điều chỉnh các tham số tăng cường phù hợp với bộ dữ liệu cụ thể là rất quan trọng.

5 HIỆN THỰC MÔ HÌNH

Nhóm sẽ sử dụng mô hình VGG16 với trọng số được lấy từ tập ImageNet (`weights='imagenet'`). Dựa vào mô hình này, nhóm sẽ áp dụng hai hướng tiếp cận khác nhau để huấn luyện:

Hướng tiếp cận với pretraining (Sử dụng mô hình pretrain)

Trong hướng tiếp cận này, nhóm sẽ sử dụng các trọng số đã được huấn luyện trước trên tập ImageNet, giúp mô hình VGG16 học được các đặc trưng hình ảnh cơ bản như cạnh, đường viền, màu sắc, v.v. từ dữ liệu ImageNet. Chỉ các lớp header (lớp cuối cùng) sẽ được huấn luyện lại, trong khi các lớp convolutional của mô hình sẽ được đóng băng (freeze), tức là không thay đổi trong quá trình huấn luyện. Điều này giúp giảm thời gian huấn luyện và tăng khả năng mô hình học được các đặc trưng chung, đồng thời giảm thiểu khả năng overfitting, nhất là khi tập dữ liệu mới không lớn.

Hướng tiếp cận không sử dụng pretraining (Không sử dụng mô hình pretrain)

Trong hướng tiếp cận này, nhóm vẫn sẽ sử dụng các trọng số đã được huấn luyện trước trên tập ImageNet nhưng cho phép huấn luyện lại các trọng số đó. Điều này có nghĩa là tất cả các lớp trong mô hình (bao gồm cả các lớp convolutional) sẽ được huấn luyện lại từ đầu. Việc này giúp mô hình có thể học các đặc trưng chuyên biệt hơn từ tập dữ liệu của nhóm, nhưng cũng đồng thời yêu cầu thời gian huấn luyện lâu hơn và dễ bị overfitting nếu tập dữ liệu huấn luyện không đủ lớn hoặc không đa dạng.

5.1 Hướng tiếp cận với pretraining (Sử dụng mô hình pretrain)

Xây dựng model VGG16 sử dụng pretrain

```
1 from tensorflow.keras import models, Sequential
2 from tensorflow.keras.layers import Flatten, Dense, Dropout
3 import tensorflow as tf
4
5 base_model = tf.keras.applications.VGG16(weights='imagenet', include_top=False,
6     input_shape=(224, 224, 3)) # Load VGG16 model pre-trained on ImageNet
7 base_model.trainable = False
8
9 model = Sequential()
10 model.add(base_model)
```



```

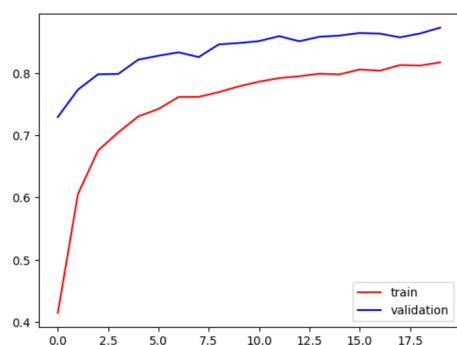
11 model.add(Flatten())
12 model.add(Dense(256, activation='relu'))
13 model.add(Dropout(0.1))
14
15 model.add(Dense(120, activation='relu'))
16 model.add(Dropout(0.1))
17
18 model.add(Dense(32, activation='relu'))
19 model.add(Dropout(0.1))
20
21 model.add(Dense(10, activation='softmax'))
22
23 optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
24 model.compile(optimizer=optimizer,
25               loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
26               metrics=['accuracy'])
27
28 model.build((None, 224, 224, 3))
29 model.summary(expand_nested=True)

```

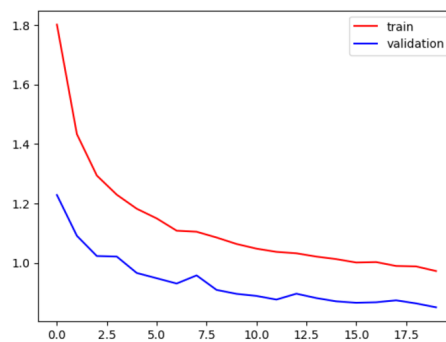
Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
input_layer_3 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten_2 (Flatten)	(None, 25088)	0
dense_9 (Dense)	(None, 256)	6,422,784
dropout_3 (Dropout)	(None, 256)	0
dense_10 (Dense)	(None, 120)	30,840
dropout_4 (Dropout)	(None, 120)	0
dense_11 (Dense)	(None, 32)	3,872
dropout_5 (Dropout)	(None, 32)	0
dense_12 (Dense)	(None, 10)	330

Bảng 3: Model Summary

Total params: 21,172,514 (80.77 MB)
Trainable params: 6,457,826 (24.63 MB)
Non-trainable params: 14,714,688 (56.13 MB)
*** Kết quả train sau khi qua 20 epoach**



Hình 30: Biểu đồ accuracy qua 20 epoach



Hình 31: Biểu đồ loss qua 20 epoach

66/66 ————— 18s 261ms/step

Accuracy: 0.8728

Classification Report:

	precision	recall	f1-score	support
butterfly	0.94	0.95	0.94	361
cat	0.85	0.84	0.84	333
chicken	0.89	0.91	0.90	517
cow	0.75	0.88	0.81	373
dog	0.89	0.82	0.85	517
elephant	0.87	0.85	0.86	289
horse	0.87	0.91	0.89	524
sheep	0.85	0.73	0.79	364
spider	0.96	0.95	0.95	517
squirrel	0.85	0.83	0.84	372
accuracy			0.87	4167
macro avg	0.87	0.87	0.87	4167
weighted avg	0.87	0.87	0.87	4167

Hình 32: Báo cáo phân loại

5.2 Hướng tiếp cận không sử dụng pretraining (Không sử dụng mô hình pretrain)

Xây dựng model VGG16 sử dụng non-pretrain

```
1 from tensorflow.keras import models, Sequential
2 from tensorflow.keras.layers import Flatten, Dense, Dropout
3 import tensorflow as tf
4
5 base_model = tf.keras.applications.VGG16(weights='imagenet', include_top=False,
6     input_shape=(224, 224, 3))
7 base_model.trainable = True
8
9 model = Sequential()
10 model.add(base_model)
11 model.add(Flatten())
12 model.add(Dense(256, activation='relu'))
13 model.add(Dropout(0.1))
14
15 model.add(Dense(120, activation='relu'))
16 model.add(Dropout(0.1))
17
18 model.add(Dense(32, activation='relu'))
19 model.add(Dropout(0.1))
20
21 model.add(Dense(10, activation='softmax'))
22
23 optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
24 model.compile(optimizer=optimizer,
25     loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1),
26     metrics=['accuracy'])
27
28 model.build((None, 224, 224, 3))
29 model.summary(expand_nested=True)
```

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
input_layer_6 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_12 (Dense)	(None, 256)	6,422,784
dropout_9 (Dropout)	(None, 256)	0
dense_13 (Dense)	(None, 120)	30,840
dropout_10 (Dropout)	(None, 120)	0
dense_14 (Dense)	(None, 32)	3,872
dropout_11 (Dropout)	(None, 32)	0
dense_15 (Dense)	(None, 10)	330

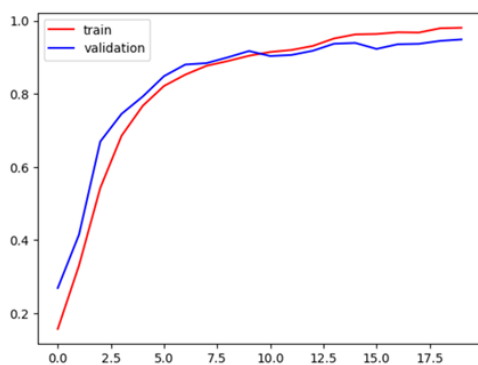
Bảng 4: Model Summary

Total params: 21,172,514 (80.77 MB)

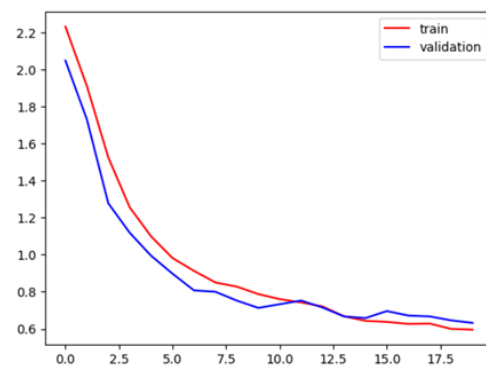
Trainable params: 21,172,514 (80.77 MB)

Non-trainable params: 0 (0.00 B)

*** Kết quả train sau khi qua 20 epoch**



Hình 33: Biểu đồ accuracy qua 20 epoch



Hình 34: Biểu đồ loss qua 20 epoch



66/66  15s 227ms/step
Accuracy: 0.9549
Classification Report:

	precision	recall	f1-score	support
butterfly	0.96	0.97	0.97	361
cat	0.95	0.95	0.95	333
chicken	0.97	0.97	0.97	517
cow	0.90	0.97	0.93	373
dog	0.95	0.93	0.94	517
elephant	0.92	0.98	0.95	289
horse	0.96	0.96	0.96	524
sheep	0.95	0.91	0.93	364
spider	0.99	0.97	0.98	517
squirrel	0.96	0.95	0.96	372
accuracy			0.95	4167
macro avg	0.95	0.95	0.95	4167
weighted avg	0.96	0.95	0.95	4167

Hình 35: Báo cáo phân loại

6 ĐÁNH GIÁ

So sánh, nhận xét và đánh giá Pretrain và Non-pretrain

1. Hiệu suất mô hình

	VGG16 non-pretrain	VGG16 pretrain
Độ chính xác (Accuracy)	0.9549	0.8639
Hàm mất mát (Loss)	0.6190	0.8576
Tốc độ hội tụ	Khoảng 5-8 epochs	Khoảng 11-13 epochs

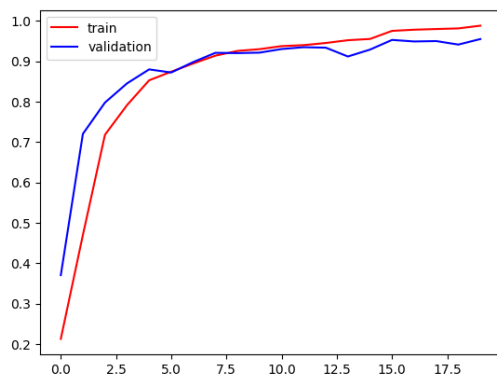
- Mô hình VGG16 non-pretrain đạt độ chính xác 0.9549, cao hơn so với VGG16 pretrain (0.8639).
- Hàm mất mát của VGG16 non-pretrain thấp hơn (0.6190 so với 0.8576) -> khả năng tối ưu hóa tốt hơn.

2. Thời gian

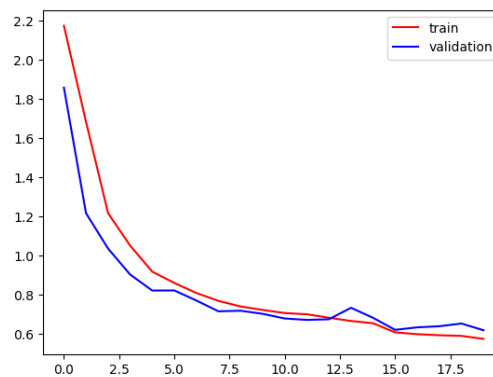
- VGG16 non-pretrain: khoảng 284.45s / 1 epoch
- VGG16 pretrain: khoảng 267.55s / 1 epoch

3. Biểu đồ

- VGG16 non-pretrain:

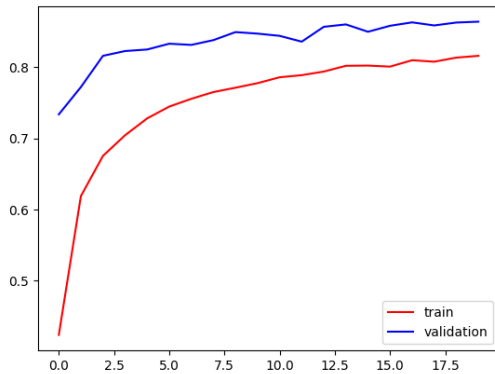


Hình 36

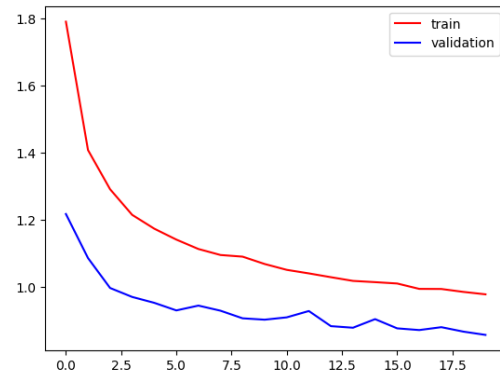


Hình 37

- VGG16 pretrain:



Hình 38



Hình 39

- Biểu đồ VGG16 non-pretrain cho thấy đường học tập (learning curve) ổn định hơn, hội tụ nhanh và đạt hiệu suất cao.
- Biểu đồ VGG16 pretrain có xu hướng dao động nhiều hơn ở các epoch, có thể do quá trình học từ trọng số ban đầu chưa phù hợp.

4. Khả năng tổng quát hóa với dữ liệu lớn hơn (Khoảng 1.3 lần dữ liệu ban đầu và số lượng ảnh trong mỗi class phân bố không đều)

	VGG16 non-pretrain	VGG16 pretrain
Độ chính xác (Accuracy)	0.9608	0.8710
Hàm mất mát (Loss)	0.6064	0.8364
Tốc độ hội tụ	Khoảng 8-10 epochs	Khoảng 13-15 epochs
Thời gian trung bình	~339.9s / 1 epoch	~322.85s / 1 epoch

Hiệu suất ở cả hai mô hình nhìn chung vẫn giữ được tính ổn định, mô hình VGG16 non-pretrain vẫn giữ được ưu thế về độ chính xác có thể do khả năng dữ liệu được huấn luyện không tương tự với dữ liệu pretrain, nên khi huấn luyện từ đầu có thể đạt được kết quả tốt hơn.

5. Nhận xét và kết luận

- VGG16 non-pretrain hiệu quả hơn trên cả hai tập dữ liệu nhờ khả năng học đặc trưng từ đầu, đặc biệt khi dữ liệu đủ lớn.
- VGG16 pretrain chỉ phù hợp khi tập dữ liệu nhỏ hoặc tương tự với dữ liệu gốc được sử dụng để pretrain.
- Dữ liệu không cân bằng cần áp dụng kỹ thuật xử lý phù hợp.
- Non-pretrain mất nhiều thời gian hơn cho mỗi epoch, nhưng bù lại đạt độ chính xác tốt hơn với số epoch ít hơn.

6. Đánh giá

• Khi nào nên sử dụng pretrain model?

Pretrain model nên được sử dụng khi tập dữ liệu nhỏ hoặc không đa dạng, khi bài toán tương tự với bài toán đã sử dụng để huấn luyện pretrain hoặc khi muốn giảm thời gian và tài nguyên huấn luyện.

• Freeze là gì?

Freeze là kỹ thuật cố định trọng số của một hoặc nhiều lớp trong mô hình, không cập nhật các trọng số này trong quá trình huấn luyện.

- **Khi nào nên freeze?**

Nên freeze khi sử dụng pretrain model để giữ lại các đặc trưng chung từ dữ liệu gốc, đặc biệt là khi dữ liệu mới nhỏ hoặc thiếu đa dạng; Khi các đặc trưng đã học của pretrained model là đủ; Khi muốn thử nghiệm tốc độ huấn luyện nhanh hơn.

7 PHƯƠNG HƯỚNG

Dựa trên các kết quả phân tích từ hai mô hình, chúng tôi đề xuất một số phương hướng giải quyết để cải thiện hiệu suất của mô hình tự xây dựng trong tương lai như sau:

- **Tăng cường dữ liệu (Data Augmentation):** Áp dụng các kỹ thuật tăng cường dữ liệu như xoay, lật, thay đổi độ sáng, và cắt xén ảnh có thể giúp tăng cường sự đa dạng của tập dữ liệu huấn luyện, từ đó cải thiện khả năng tổng quát của mô hình.
- **Tối ưu hóa siêu tham số (Hyperparameter Optimization):** Thực hiện các nghiên cứu để tối ưu hóa các siêu tham số của mô hình, chẳng hạn như kích thước batch, tốc độ học (learning rate), và số lượng lớp và nút trong các lớp fully connected có thể giúp cải thiện độ chính xác.
- **Phân tích và điều chỉnh dữ liệu đầu vào (Data Input Analysis and Adjustment):** Kiểm tra và điều chỉnh cách thức thu thập và xử lý dữ liệu đầu vào để đảm bảo rằng mô hình được huấn luyện trên dữ liệu có chất lượng cao và đại diện cho bài toán.
- **Đánh giá và cải thiện mô hình (Model Evaluation and Improvement):** Thiết lập các phương pháp đánh giá liên tục để theo dõi hiệu suất của mô hình trên tập dữ liệu mới, từ đó điều chỉnh và cải thiện mô hình một cách kịp thời.

8 TÀI LIỆU THAM KHẢO

- [1] <https://viblo.asia/p/cac-phuong-phap-tranh-overfitting-gDVK24AmlLj>
- [2] <https://nttuan8.com/bai-6-convolutional-neural-network/>
- [3] <https://www.geeksforgeeks.org/adam-optimizer/>
- [4] <https://cv-tricks.com/keras/understand-implement-resnets/>
- [5] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. arXiv link
- [6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun: Identity Mappings in Deep Residual Networks arXiv link